

Classification

Assigned reading:

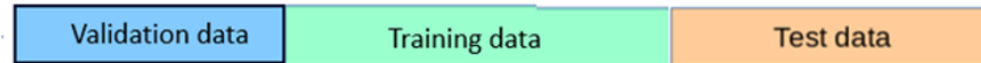
- 5.0 – 5.1.4 Classification
- 5.3 up to 5.3.3 Generative Classifiers

Next class:

- 5.4-5.4.4 Discriminative Classifiers, Logistic Regression
- 5.2.5, 5.2.6 Classifier Accuracy, ROC

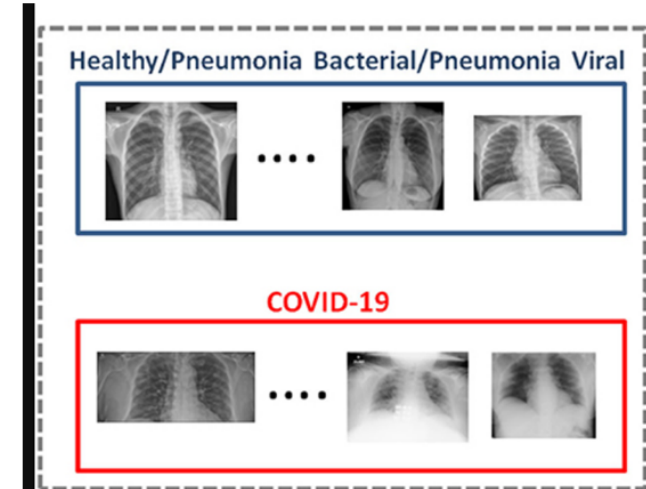
Recall from last class:

Use of train vs test sets



Always need to do *model selection*:

- What model class to use? Linear regression, neural network?
 - Picking which features to use in linear regression.
 - Picking which ridge penalty (*hyperparameter*) to use, λ .
 - For neural networks—picking what architecture to choose.
 - *Model selection* cannot be done by MLE/optimization.
- We must use train/test/validation splits to choose the *hyper-parameters*



Role of train vs test (e.g. Hospital xray prediction)

- Use validation set to pick model and/or hyper-parameters.
- Through “winner’s curse” will get lower error estimate on validation set than is real.
- After picking model & hyper-parameters, use test set ONCE to get unbiased error estimate.

From lecture 3: UCB paper about train/test splits

Do ImageNet Classifiers Generalize to ImageNet?

Benjamin Recht* Rebecca Roelofs Ludwig Schmidt Vaishaal Shankar
UC Berkeley UC Berkeley UC Berkeley UC Berkeley

Abstract

We build new test sets for the CIFAR-10 and ImageNet datasets. Both benchmarks have been the focus of intense research for almost a decade, raising the danger of overfitting to excessively re-used test sets. By closely following the original dataset creation processes, we test to what extent current classification models generalize to new data. We evaluate a broad range of models and find accuracy drops of 3% – 15% on CIFAR-10 and 11% – 14% on ImageNet. However, accuracy gains on the original test sets translate to larger gains on the new test sets. Our results suggest that the accuracy drops are not caused by adaptivity, but by the models' inability to generalize to slightly "harder" images than those found in the original test sets.

*Authors ordered alphabetically. Ben did none of the work.

ImageNetV2 was **collected in the same way** as ImageNet but ImageNet models **achieve a lower accuracy**

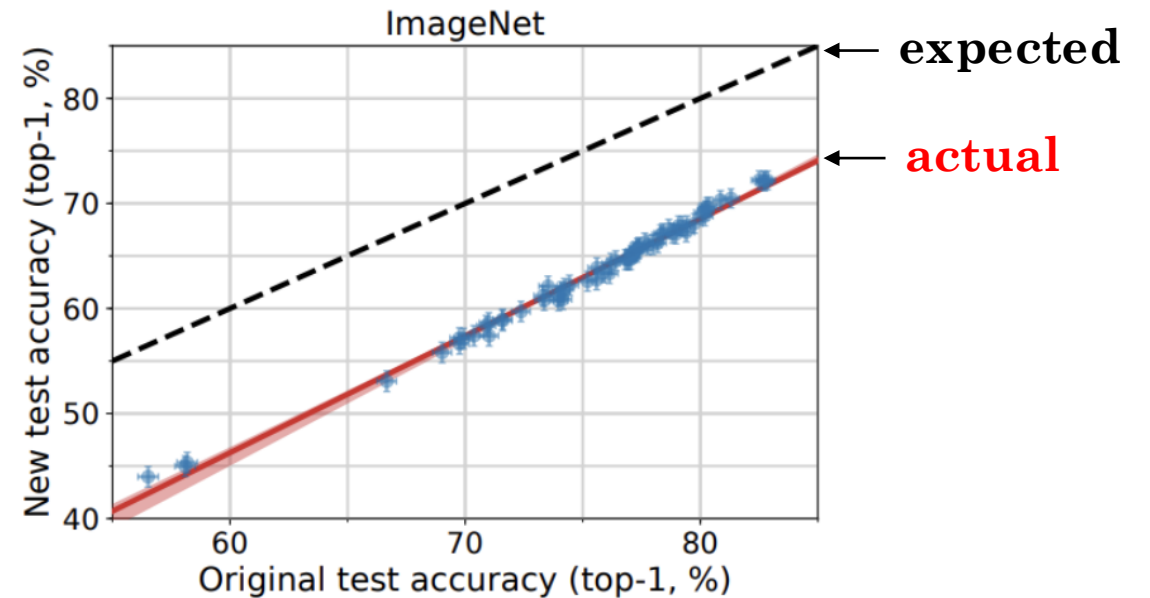


ImageNet

ImageNetV2

visually very similar

ImageNetV2 was **collected in the same way** as ImageNet but ImageNet models **achieve a lower accuracy**



CS 189/289

Today's lecture outline:

1. Discriminative vs. Generative Classification
2. Gaussian Discriminant Analysis
3. Evaluating Classifiers

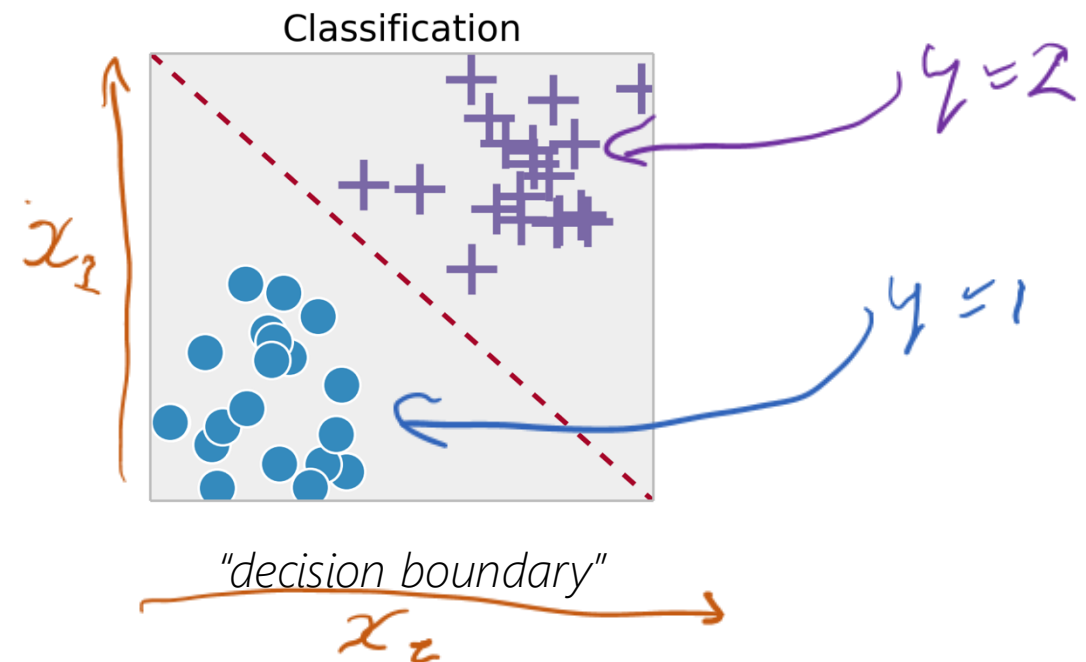
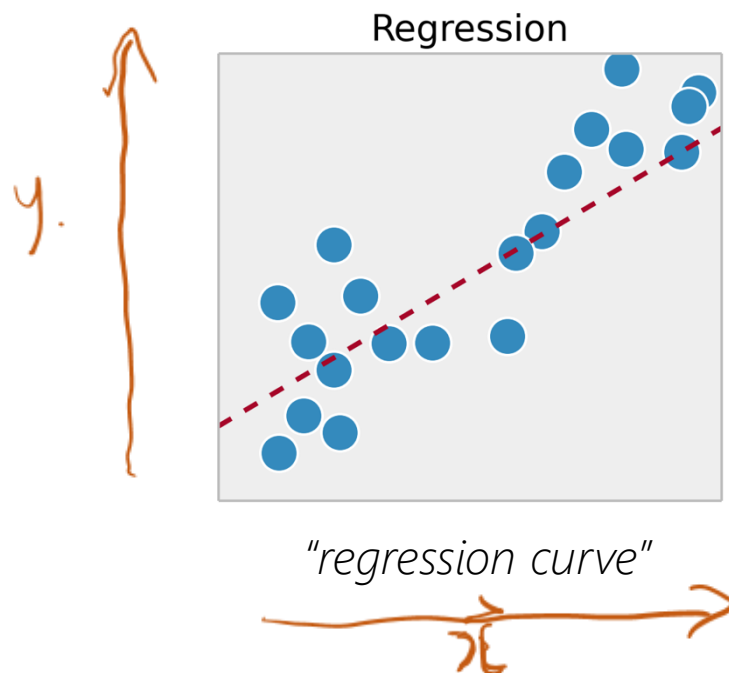
CS 189/289

Today's lecture outline:

1. Discriminative vs. Generative Classification
2. Gaussian Discriminant Analysis
3. Evaluating Classifiers

Recall: classification vs. regression

- Regression: try to “draw a line to trace out the data”.
- Classification: try to “draw a line to separate the classes of data”.
 - i.e. separate with a *decision boundary*: further away point is, the more confident the prediction.



Three strategies to build a classifier

1. Generative probabilistic models (indirectly *via* Bayes Rule):

$$p(C_k|x) = \frac{p(x|C_k)p(C_k)}{p(x)}$$

probabilistic
classifiers

2. Discriminative probabilistic models (directly):

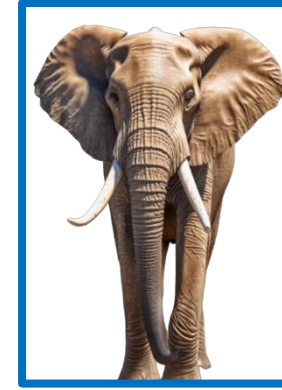
$$p(C_k|x)$$

3. Discriminant functions: Direct assignment to a class, $f(x) \rightarrow k$.

What are advantages of going probabilistic here?

Two fundamental types of probabilistic classifiers

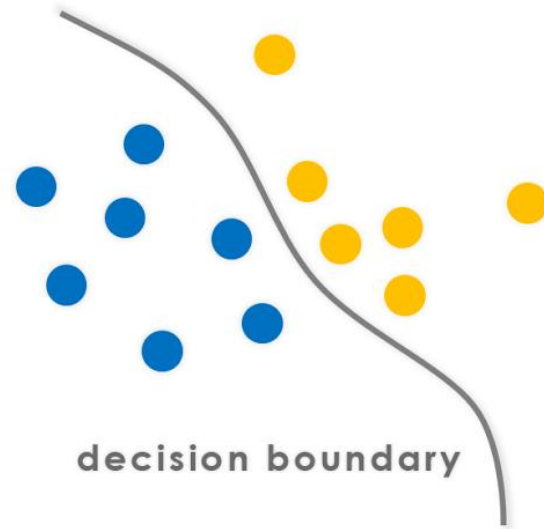
- Discriminative vs Generative Classifiers
- What do these mean?



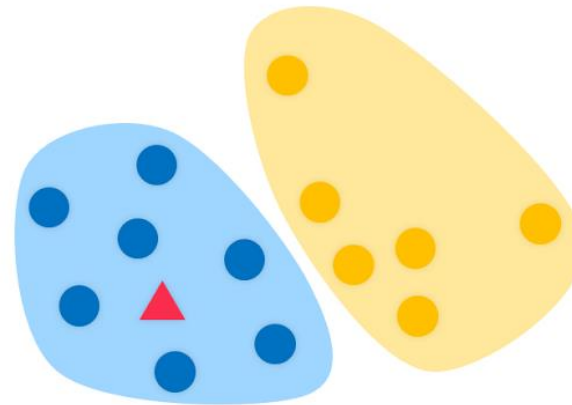
VS



Discriminative



Generative



A tale of two children (discriminative vs generative)

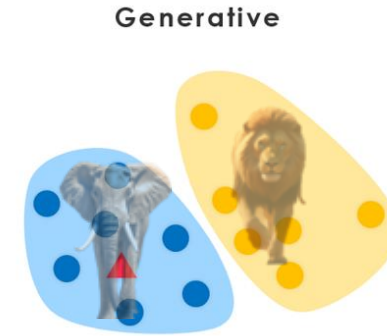
A parent has two children:

- Child A has a knack for learning everything in a holistic manner.
 - Child B narrowly focuses on the most pertinent information needed, discarding the rest.
- 
- At the zoo, they are instructed to observe the two animals in order to tell them apart.
 - Later that night, they are watching a movie, and the parent asks the children: "is this animal we see on the screen a lion or an elephant?"

What are two different ways these children might have learned to build a lion-elephant classifier in their heads?

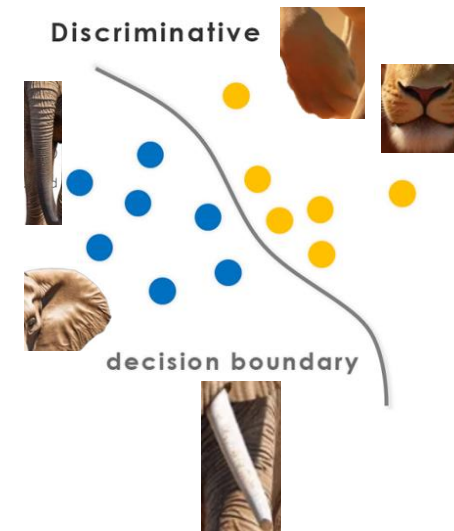
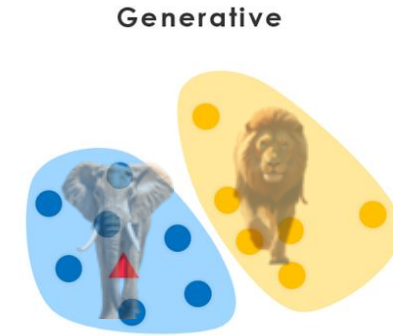
A tale of two children (discriminative vs generative)

- Child A had drawn two images, one of a lion and one of an elephant on a piece of paper. Then compares each image to the animal on the TV, and answers, "the animal is Lion".

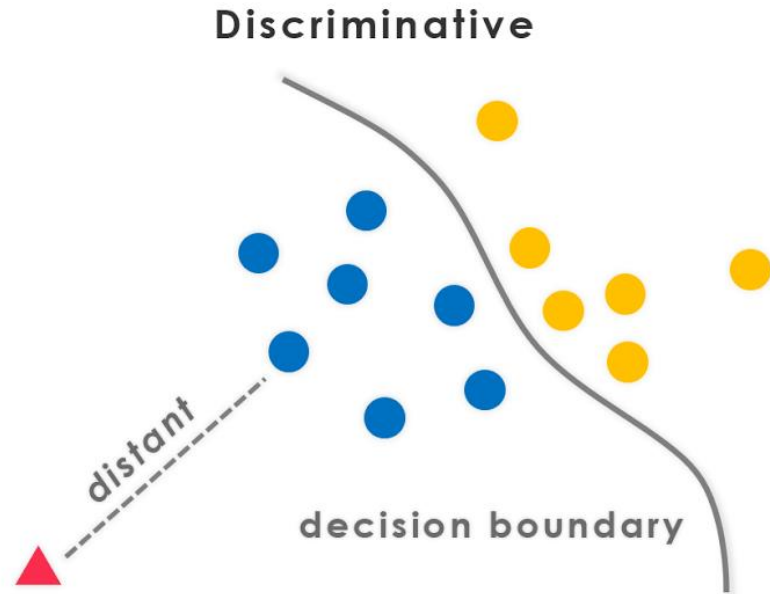


A tale of two children (discriminative vs generative)

- Child A had drawn two images, one of a lion and one of an elephant on a piece of paper. Then compares each image to the animal on the TV, and answers, "the animal is Lion".
- Child B chose not to remember what each animal looked like, but remembers the differences in some of their features (e.g. trunk or no trunk) and answers: "the animal is a Lion".

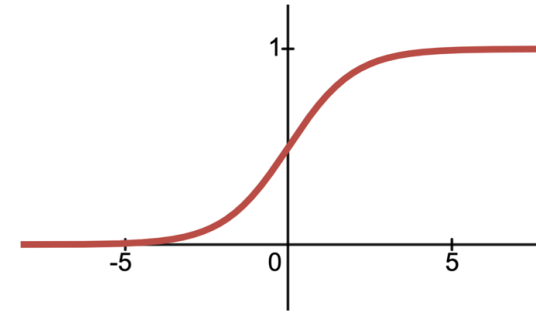


Discriminative vs Generative Classifiers



$$p(y|X)$$

e.g., $p(y = C|X, w) = \text{sigmoid}(w^T x)$

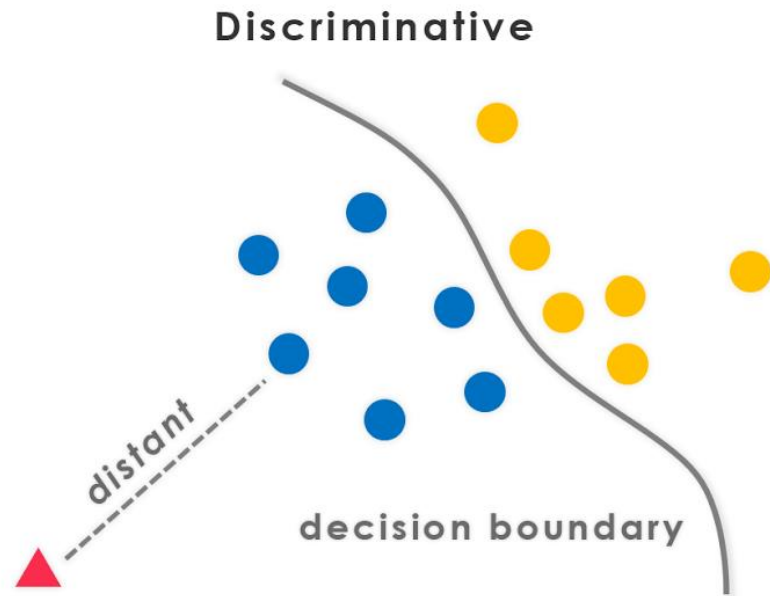


$$\sigma(a) = \frac{1}{1 + e^{-a}}$$

sigmoid function

Logistic regression

Discriminative vs Generative Classifiers

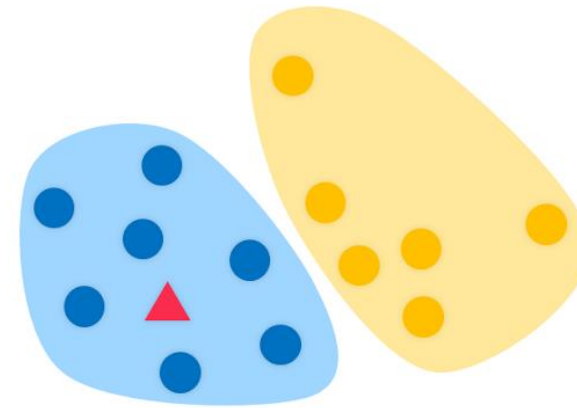


$$p(y|X)$$

e.g., $p(y = C|X, w) = \text{sigmoid}(w^T x)$

Logistic regression

Generative



$$p(y|X) \propto p(X, y) = p(X|y)p(y)$$

e.g., $p(X|y = C, \mu_c, \Sigma_c) = N(X|\mu_c, \Sigma_c)$

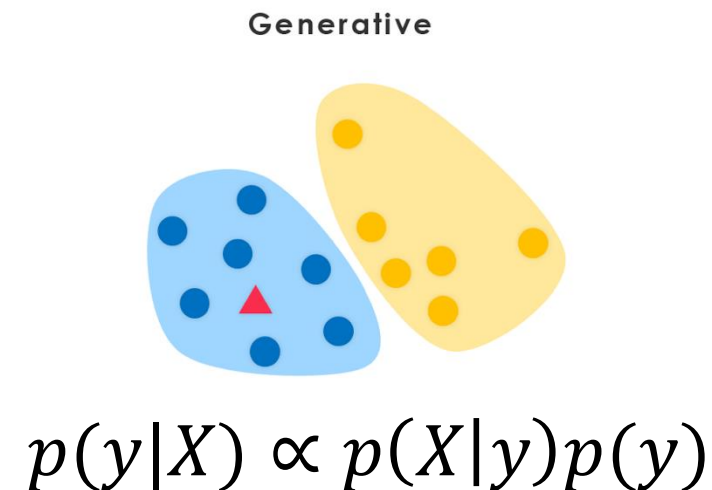
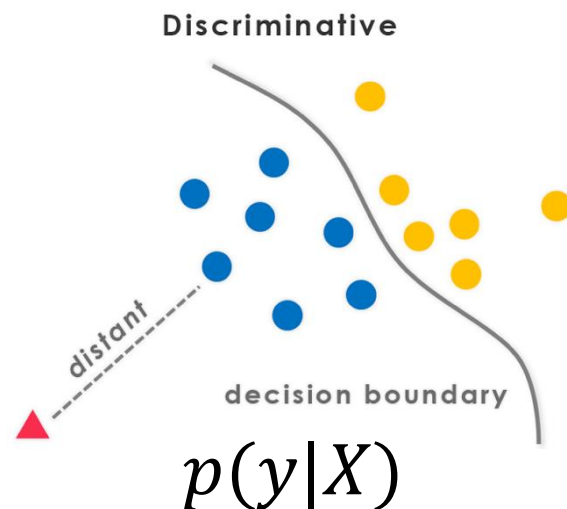
$$p(y = 1) = 0.22$$

$$p(y = 0) = 0.78$$

class-conditional gaussians

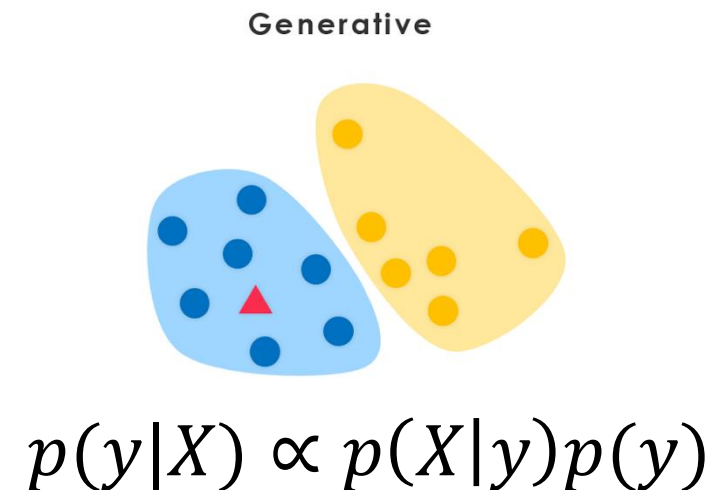
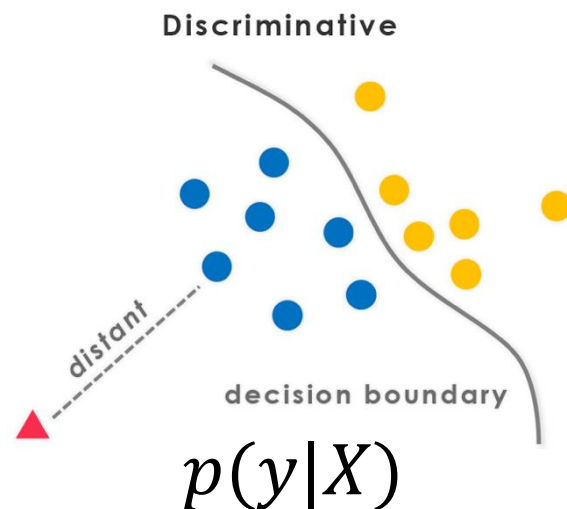
What implications does this have?

- Discriminative models “spend” all of their parameters to model the decision boundary.
- Generative models “spend” parameters to fully model fully model the density of $\mathbf{X}$ of each class, and imply a discriminative function.
- Thus, given the same capacity ($\approx$ # **params**), a discriminative model may yield decision boundaries.



What implications does this have?

- Discriminative models “spend” all of their parameters to model the decision boundary.
- Generative models “spend” parameters to fully model fully model the density of $\mathbf{X}$ of each class, and imply a discriminative function.
- Thus, given the same capacity ($\approx$ # **params**), a discriminative model may yield more complex decision boundaries.



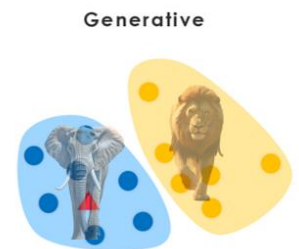
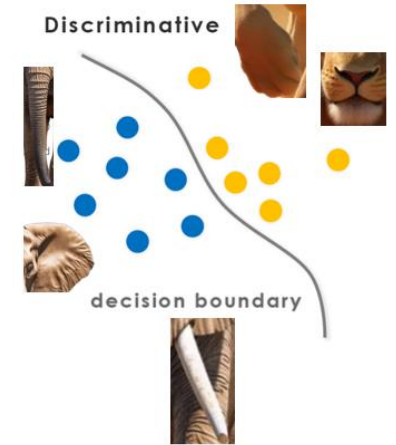
Discriminative vs Generative Classifiers

- A discriminative model directly models the discriminative function, $\text{score}(y = a) = f(X)$.
- This score may or may not be probabilistic:
 $f(X) = p(y|x)$ vs. "distance to boundary"
- A generative model is defined by the joint probability, factored as:

$$p(y = C, X) = p(X|y = C)p(y = C).$$

then uses Bayes Rule:

$$p(y = C|X) = \frac{p(X|y = C)p(y = C)}{p(X)}$$



Discriminative vs Generative Classifiers

- Generative model describes an (idealized) method of how data were generated.
- Generative models have discriminative properties (*via* Bayes Rule).
- Discriminative models do not have (full) generative properties.
- Which type makes stronger statistical assumptions?

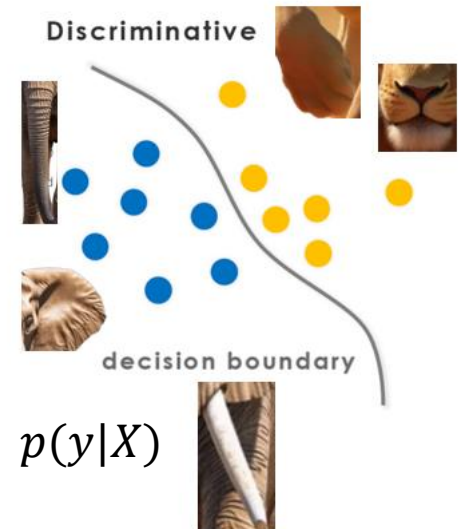
$$D = \{(x_i, y_i)\}$$

Generative



$$p(y|X) \propto p(X, y) = p(X|y)p(y)$$

Discriminative

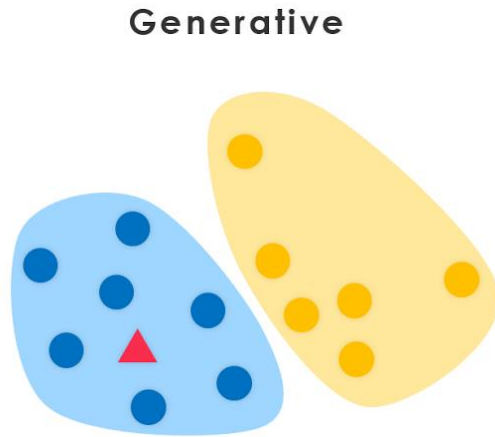


CS 189/289

Today's lecture outline:

1. Discriminative vs. Generative Classification
2. Gaussian "Discriminant" Analysis
3. Evaluating Classifiers

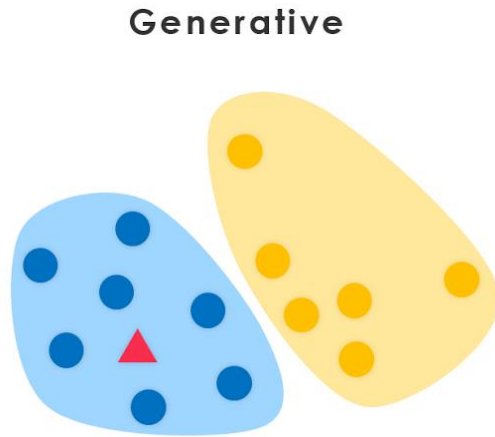
Gaussian “Discriminant” Analysis



- Generative model (not discriminative!)
- Use Gaussians to model X conditioned on the class.

$$p(X, y) = p(X|y)p(y)$$

Gaussian “Discriminant” Analysis



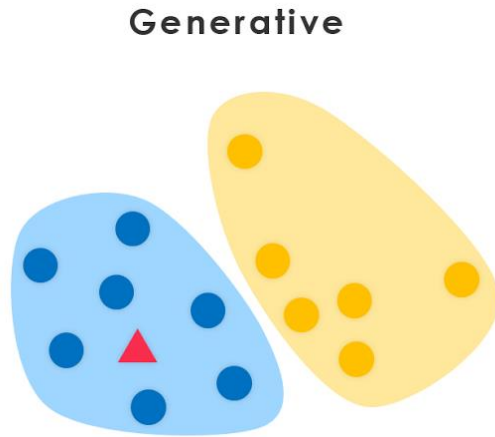
- Generative model (not discriminative!)
- Use Gaussians to model X conditioned on the class.

$$p(X, y) = p(X|y)p(y)$$

$$p(X|y = 1, \mu_1, \Sigma_1) = N(X|\mu_1, \Sigma_1)$$

$$p(X|y = 0, \mu_0, \Sigma_0) = N(X|\mu_0, \Sigma_0)$$

Gaussian “Discriminant” Analysis



- Generative model.
- Use Gaussians to model X conditioned on the class.

$$p(X, y) = p(X|y)p(y)$$

$$p(X|y = 1, \mu_1, \Sigma_1) = N(X|\mu_1, \Sigma_1)$$

$$p(X|y = 0, \mu_0, \Sigma_0) = N(X|\mu_0, \Sigma_0)$$

$$p(y = 1) = \text{Bernoulli}(\phi) = 0.22$$

$$p(y = 0) = \text{Bernoulli}(1 - \phi) = 0.78$$

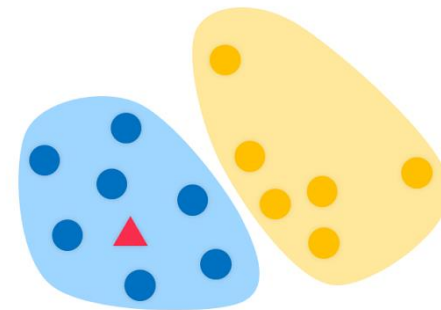
Gaussian Discriminant Analysis

What are the choices could we make:

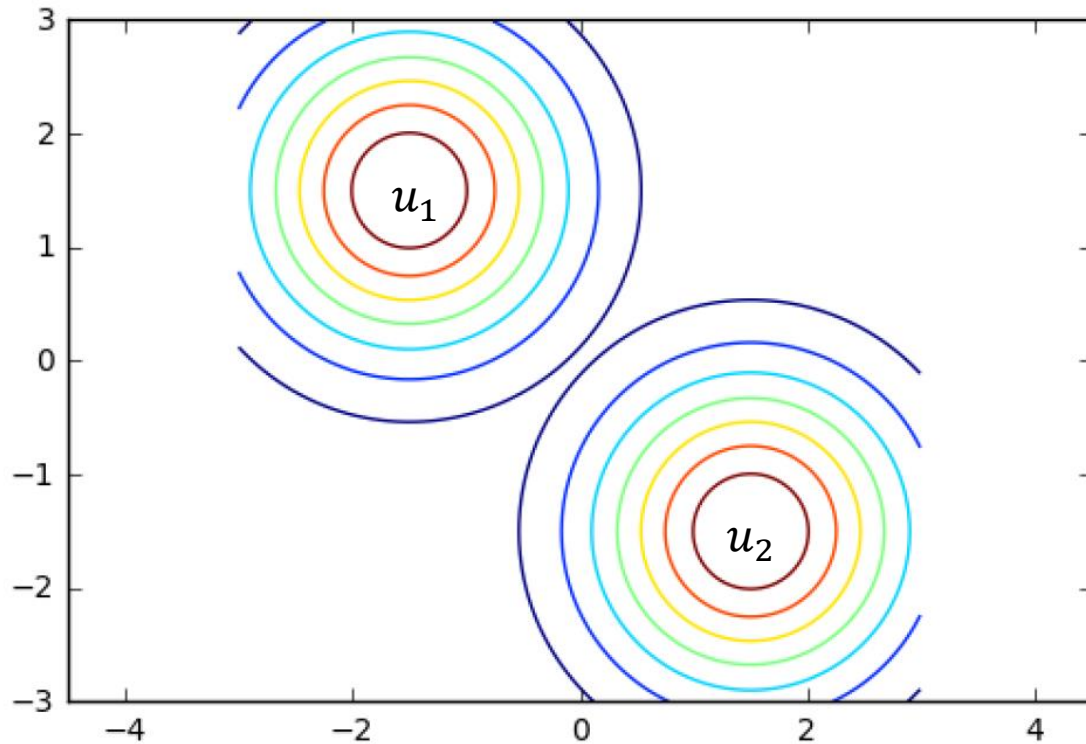
- Same/different mean parameters for each class?
- Same/different covariance parameters for each class?
- Should covariance be diagonal? Spherical?

$$p(X|y = 1, \mu_1, \Sigma_1) = N(X|\mu_1, \Sigma_1)$$
$$p(X|y = 0, \mu_0, \Sigma_0) = N(X|\mu_0, \Sigma_0)$$

Generative



Gaussian Discriminant Analysis

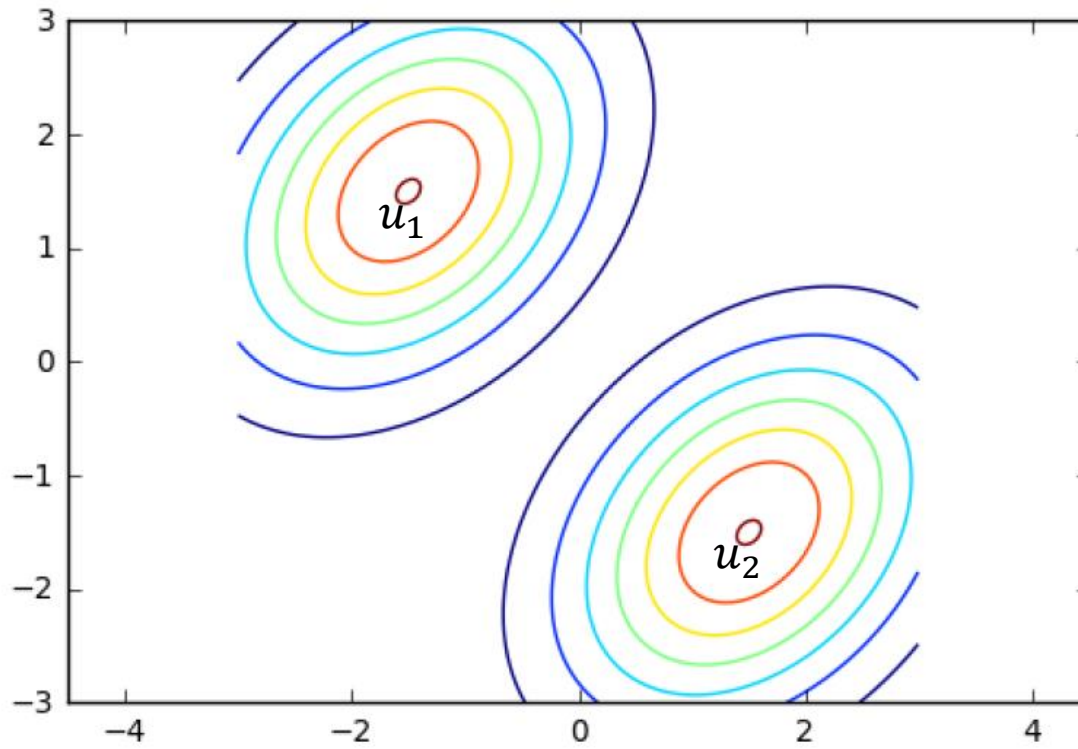


We will always assume that we want different means, u_1 and u_2 .

Case 1: two identical, “spherical” (isotropic) Gaussians:

$$\Sigma_1 = \Sigma_2 = \Sigma = \text{diag}(\sigma, \sigma, \dots, \sigma)$$

Gaussian Discriminant Analysis

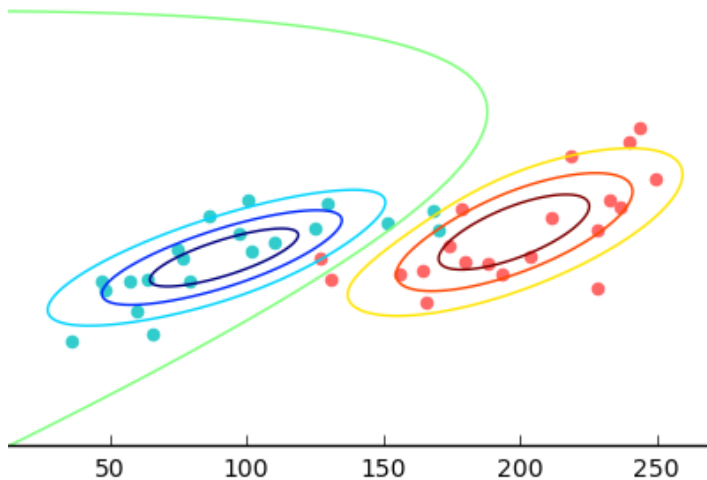


Case 2: two identical, arbitrary covariance (anisotropic) Gaussians:

$$\Sigma_1 = \Sigma_2 = \Sigma$$

Gaussian Discriminant Analysis

Most general case: means and covariance can be different for each class, and covariance is arbitrary:



$$p(X|y = 1, \mu_1, \Sigma_1) = N(X|\mu_1, \Sigma_1)$$

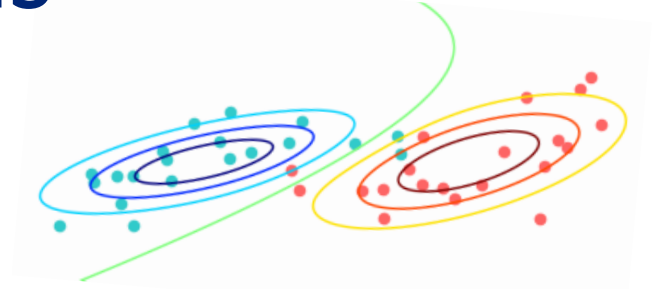
$$p(X|y = 0, \mu_0, \Sigma_0) = N(X|\mu_0, \Sigma_0)$$

Parameters: $\{\mu_0, \Sigma_0, \mu_1, \Sigma_1\}$

All of these variants readily generalize to $K > 2$ classes.

Gaussian Discriminant Analysis

- How to do parameter estimation?
- Maximum likelihood estimation (MLE).
- Data from k classes, $X \in R^{N \times D}$ and $Y \in \{1, \dots, K\}^N$.
- What will MLE loss be here?



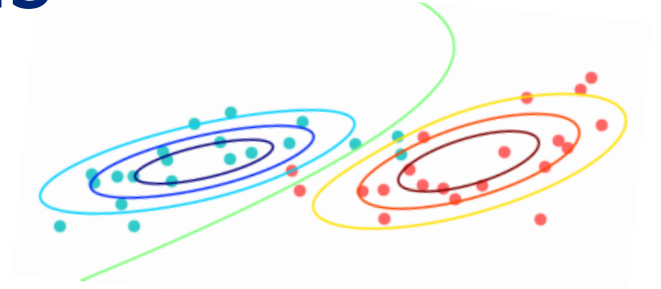
$$p(y = c|X) = \frac{p(X|y = c)p(y = c)}{p(X)}$$

$$p(y|X) \propto p(X, y) = p(X|y)p(y)$$

$$\log p(\text{???} | \{\mu_k, \Sigma_k\}) = \text{???}$$

Gaussian Discriminant Analysis

- How to do parameter estimation?
- Maximum likelihood estimation (MLE).
- Data from k classes, $X \in \mathbb{R}^{N \times D}$ and $Y \in \{1, \dots, K\}^N$.
- What will MLE loss be here?
- The joint probability of the training data:



$$p(y = c|X) = \frac{p(X|y = c)p(y = c)}{p(X)}$$

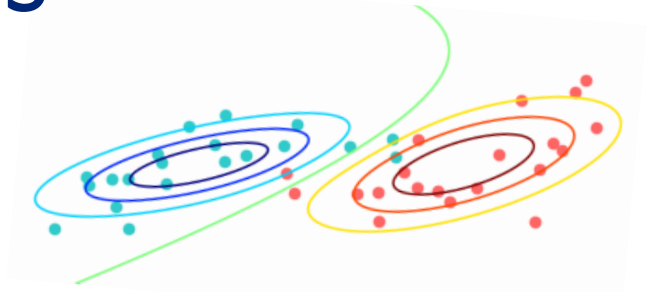
$$p(y|X) \propto p(X, y) = p(X|y)p(y)$$

$$\log p(Y, X | \{\mu_k, \Sigma_k\}) = \sum_{\text{class } k} \sum_{i \in \{i | y_i = k\}} (\log N(x_i | \mu_k, \Sigma_k) + \log p(y_i))$$

this is how you know it is a generative classifier (and not discriminative)

Gaussian Discriminant Analysis

- How to do parameter estimation?
- Maximum likelihood estimation (MLE).
- Data from k classes, $X \in R^{N \times D}$ and $Y \in \{1, \dots, K\}^N$.
- What will MLE loss be here?
- The joint probability of the training data:



$$p(y = c|X) = \frac{p(X|y = c)p(y = c)}{p(X)}$$

$$p(y|X) \propto p(X, y) = p(X|y)p(y)$$

$$\log p(Y, X | \{\mu_k, \Sigma_k\}) = \sum_{\text{class } k} \sum_{i \in \{i | y_i = k\}} (\log N(x_i | \mu_k, \Sigma_k) + \log p(y_i))$$

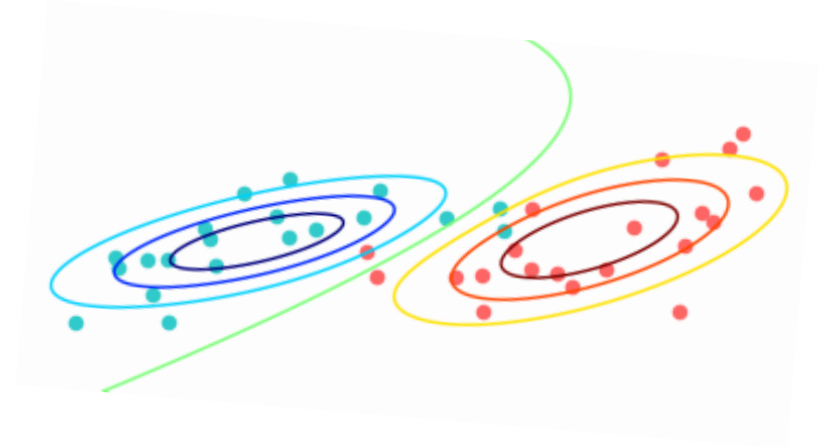
Take a second to think through MLE for u_1 ---can you see the simple & intuitive outcome?!

Gaussian Discriminant Analysis

Result (same as MLE for a Gaussian!):

$$\hat{\mu}_k = \frac{1}{n_k} \sum_{t_i=k} X_i$$

$$\hat{\Sigma}_k = \frac{1}{n_k} \sum_{t_i=k} (X_i - \hat{\mu}_k)(X_i - \hat{\mu}_k)^T$$



Log likelihood of the data:

What about setting $p(Y)$?

$$\log p(Y, X | \{\mu_k, \Sigma_k\}) = \sum_{\text{class } k} \sum_{i \in \{i | y_i = k\}} (\log N(x_i | \mu_k, \Sigma_k) + \log p(y_i))$$

Gaussian Discriminant Analysis

Can think of $p(\mathbf{y} = \mathcal{C})$ in two ways:

- a) MLE for the Bernoulli (or “Multinoulli” parameter).
- b) As a prior, specified by domain expert/knowledge (e.g. doctor who knows prevalence of Crohn’s disease).

Why do (b) instead of (a) if you have training data?

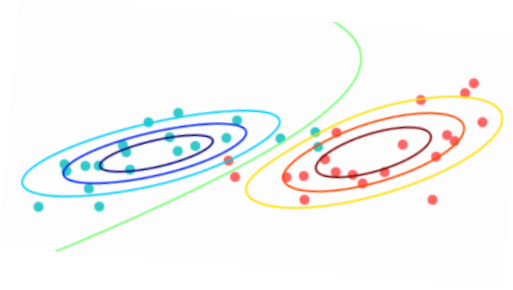
$$p(Y|X) \propto p(X|Y)p(Y)$$

Gaussian Discriminant Analysis

Now we know how to set all three kinds of parameters:

$$\hat{\mu}_k = \frac{1}{n_k} \sum_{t_i=k} X_i$$

$$\hat{\Sigma}_k = \frac{1}{n_k} \sum_{t_i=k} (X_i - \hat{\mu}_k)(X_i - \hat{\mu}_k)^T$$

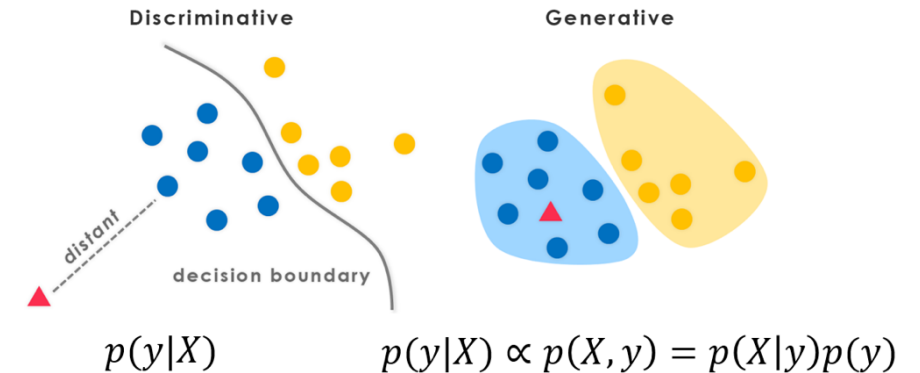


$p(Y)$ is learned from data, or set as a prior

$$p(Y|X) \propto p(X|Y)p(Y)$$

Gaussian Discriminant Analysis

- Lets take a closer look at the decision boundary implied by GDA.
- Formally, the decision boundary between class a and b is defined as ...?

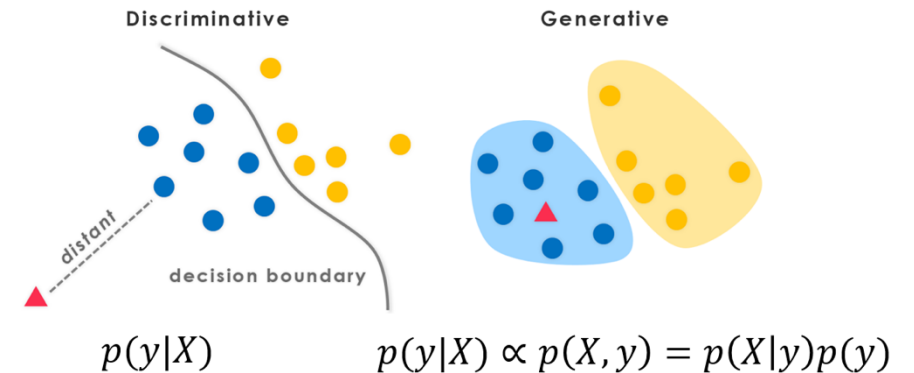


Gaussian Discriminant Analysis

- Lets take a closer look at the decision boundary implied by GDA.
- Formally, the decision boundary between class a and b is defined as:

➤ the level set in X where $p(Y = a|X) = p(Y = b|X)$
for $X \in R^{D \times 1}$ and $Y \in R^1$

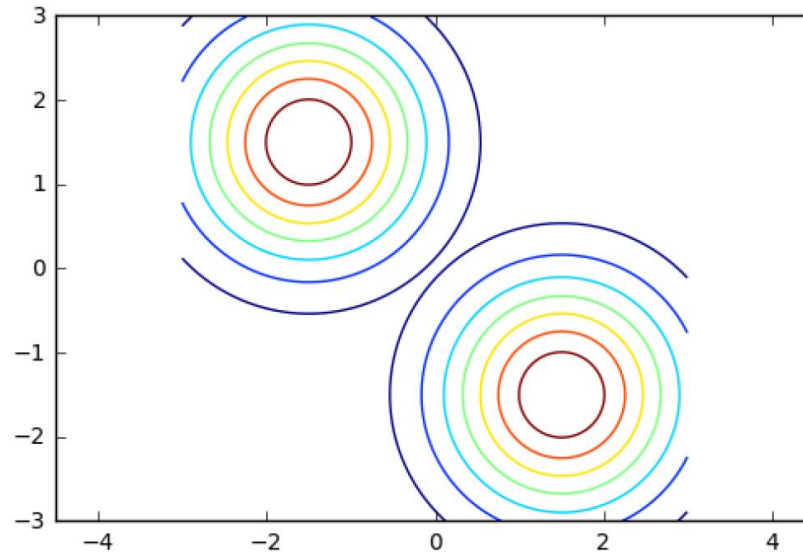
Lets use this to get more insights.



GDA Decision Boundary

- Assume equal class priors for each class, $p(a) = p(b)$.
- Assume identical, isotropic Gaussians:

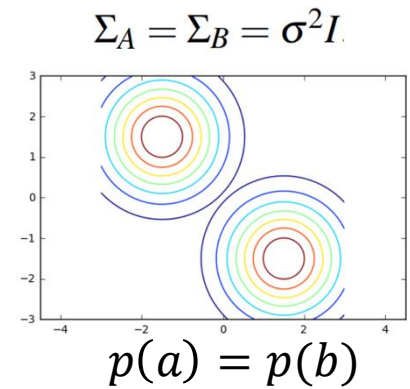
$$\Sigma_A = \Sigma_B = \sigma^2 I.$$



GDA Decision Boundary

Set class predictions to be the same:

$$p(Y = a|X) = p(Y = b|X)$$

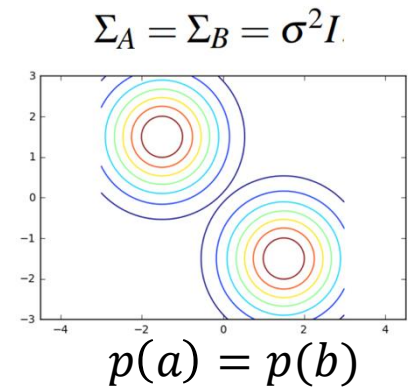


GDA Decision Boundary

Set class predictions to be the same:

$$p(Y = a|X) = p(Y = b|X)$$

$$p(X|Y = a)p(Y = a) = p(X|y = b)p(y = b)$$



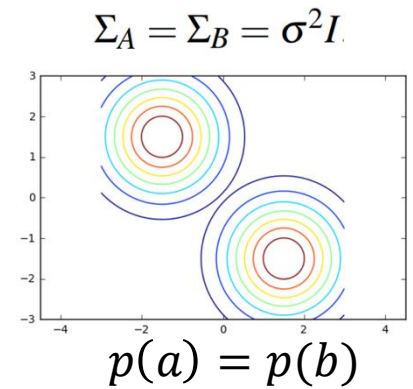
GDA Decision Boundary

Set class predictions to be the same:

$$p(Y = a|X) = p(Y = b|X)$$

$$p(X|Y = a)p(Y = a) = p(X|y = b)p(y = b)$$

$$p(X|Y = a) = p(X|y = b)$$



GDA Decision Boundary

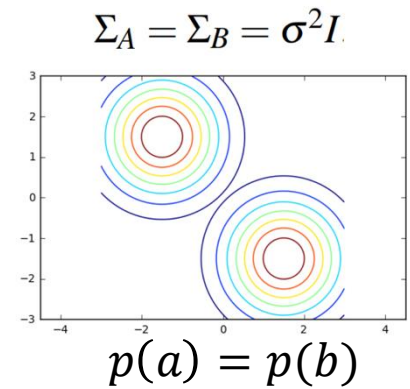
Set class predictions to be the same:

$$p(Y = a|X) = p(Y = b|X)$$

$$p(X|Y = a)p(Y = a) = p(X|y = b)p(y = b)$$

$$p(X|Y = a) = p(X|y = b)$$

$$N(X|\hat{\mu}_a, I\sigma^2) = N(X|\hat{\mu}_b, I\sigma^2)$$



GDA Decision Boundary

Set class predictions to be the same:

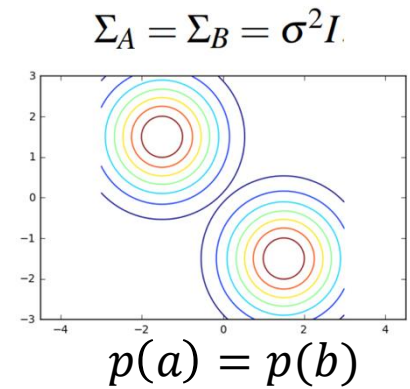
$$p(Y = a|X) = p(Y = b|X)$$

$$p(X|Y = a)p(Y = a) = p(X|y = b)p(y = b)$$

$$p(X|Y = a) = p(X|y = b)$$

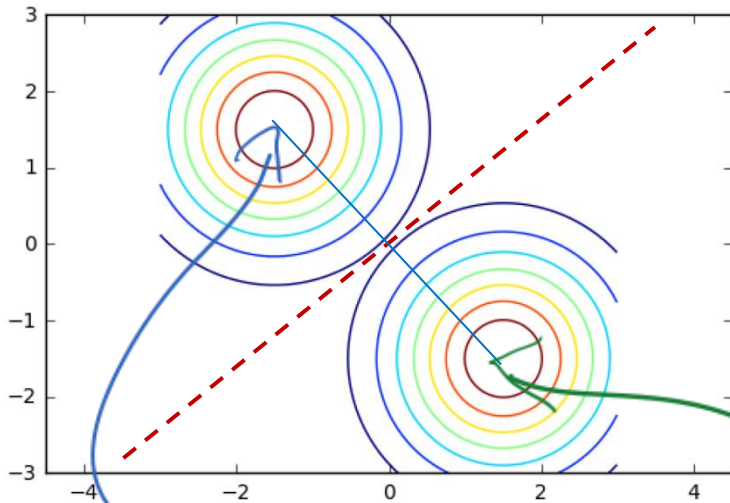
$$N(X|\hat{\mu}_a, I\sigma^2) = N(X|\hat{\mu}_b, I\sigma^2)$$

$$\log N(X|\hat{\mu}_a, I\sigma^2) = \log N(X|\hat{\mu}_b, I\sigma^2)$$



$$\begin{aligned} & \ln\left(\frac{1}{2}\right) - \frac{1}{2}(X - \hat{\mu}_A)^T \sigma^2 I (X - \hat{\mu}_A) - \frac{1}{2} \ln(|\sigma^2 I|) \\ = & \ln\left(\frac{1}{2}\right) - \frac{1}{2}(X - \hat{\mu}_B)^T \sigma^2 I (X - \hat{\mu}_B) - \frac{1}{2} \ln(|\sigma^2 I|) \end{aligned}$$

GDA Decision Boundary



Thus decision boundary is set of X for which $\|X - \hat{\mu}_A\|_2 = \|X - \hat{\mu}_B\|_2$.

$$2(\mu_B - \mu_A)^T X = \mu_B^T \mu_B - \mu_A^T \mu_A$$

i.e. points equidistant from $\hat{\mu}_a$ and $\hat{\mu}_b$.

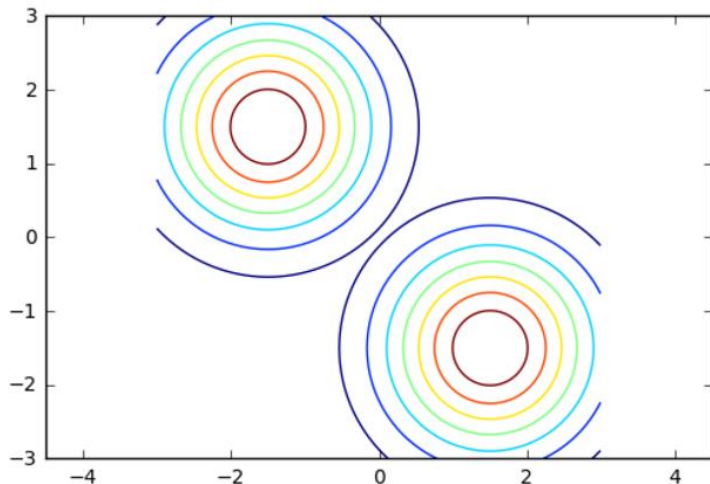
$$\Rightarrow (X - \hat{\mu}_A)^T (X - \hat{\mu}_A) = (X - \hat{\mu}_B)^T (X - \hat{\mu}_B)$$

$$\begin{aligned} & \ln\left(\frac{1}{2}\right) - \frac{1}{2}(X - \hat{\mu}_A)^T \sigma^2 I (X - \hat{\mu}_A) - \frac{1}{2} \ln(|\sigma^2 I|) \\ &= \ln\left(\frac{1}{2}\right) - \frac{1}{2}(X - \hat{\mu}_B)^T \sigma^2 I (X - \hat{\mu}_B) - \frac{1}{2} \ln(|\sigma^2 I|) \end{aligned}$$

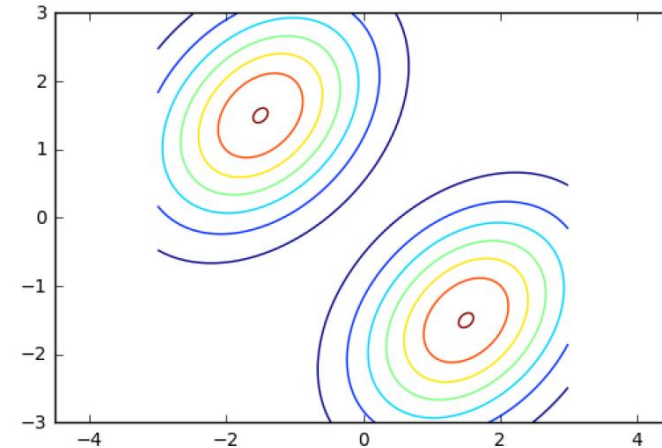
GDA Decision Boundary

- We had assumed equal class priors and equal spherical covariances.
- What happens if we let the covariances be equal, but of arbitrary shape (anisotropic)?

$$\Sigma_A = \Sigma_B = \sigma^2 I$$



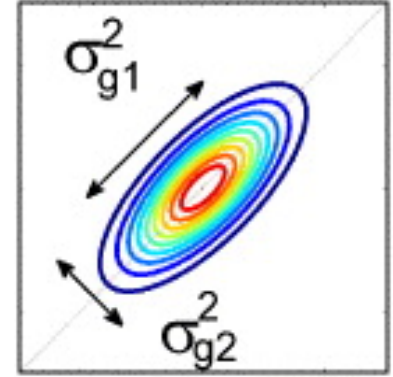
$$\Sigma_1 = \Sigma_2 = \Sigma$$



GDA Decision Boundary

- Isotropic case we get a line from: $(X - \hat{\mu}_A)^T (X - \hat{\mu}_A) = (X - \hat{\mu}_B)^T (X - \hat{\mu}_B)$
- With arbitrary covariance, get ellipsoids:

$$(X - \hat{\mu}_A)^T \hat{\Sigma}^{-1} (X - \hat{\mu}_A) = (X - \hat{\mu}_B)^T \hat{\Sigma}^{-1} (X - \hat{\mu}_B).$$



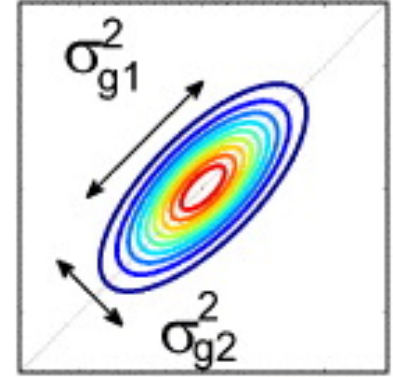
GDA Decision Boundary

- Isotropic case we get a line from: $(X - \hat{\mu}_A)^T (X - \hat{\mu}_A) = (X - \hat{\mu}_B)^T (X - \hat{\mu}_B)$
- With arbitrary covariance, get ellipsoids:

$$(X - \hat{\mu}_A)^T \hat{\Sigma}^{-1} (X - \hat{\mu}_A) = (X - \hat{\mu}_B)^T \hat{\Sigma}^{-1} (X - \hat{\mu}_B).$$

Spectral decomposition $\Sigma = U S U^T$ tells us how to rotate and scale X so that its covariance is I :

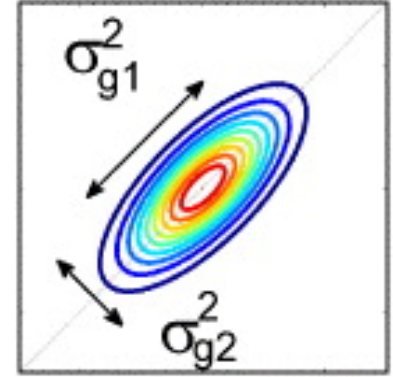
$$\hat{X} = S^{-\frac{1}{2}} U^T (X - \mu_A) \Rightarrow \text{cov}(\hat{X}) = I$$



GDA Decision Boundary

- Isotropic case we get a line from: $(X - \hat{\mu}_A)^T (X - \hat{\mu}_A) = (X - \hat{\mu}_B)^T (X - \hat{\mu}_B)$
- With arbitrary covariance, get ellipsoids:

$$(X - \hat{\mu}_A)^T \hat{\Sigma}^{-1} (X - \hat{\mu}_A) = (X - \hat{\mu}_B)^T \hat{\Sigma}^{-1} (X - \hat{\mu}_B).$$



Spectral decomposition $\Sigma = U S U^T$ tells us how to rotate and scale X so that its covariance is I :

$$\hat{X} = S^{-\frac{1}{2}} U^T (X - \mu_A) \Rightarrow \text{cov}(\hat{X}) = I$$

$$\text{Thus } (\hat{X} - \hat{\mu}_A)^T I (\hat{X} - \hat{\mu}_A) = (\hat{X} - \hat{\mu}_B)^T I (\hat{X} - \hat{\mu}_B).$$

Linear transformation to spherical, so still have line as a decision boundary!

GDA Decision Boundary

What happens if the class priors are not equal?

$$p(X|Y = a)p(Y = b) = p(X|y = b)p(y = b)$$
$$\log N(X|\hat{\mu}_a, \hat{\Sigma}) + \log p(Y = b) = \log N(X|\hat{\mu}_b, \hat{\Sigma}) + \log p(Y = b)$$

- We are adding a constant into the system.
- Leaves the decision boundary as a line.

GDA Decision Boundary

Finally, what happens if also allow different covariances for each class?

$$p(X|Y = a)p(Y = b) = p(X|y = b)p(y = b)$$
$$\log N(X|\hat{\mu}_a, \hat{\Sigma}_a) + \log p(Y = b) = \log N(X|\hat{\mu}_b, \hat{\Sigma}_b) + \log p(Y = b)$$

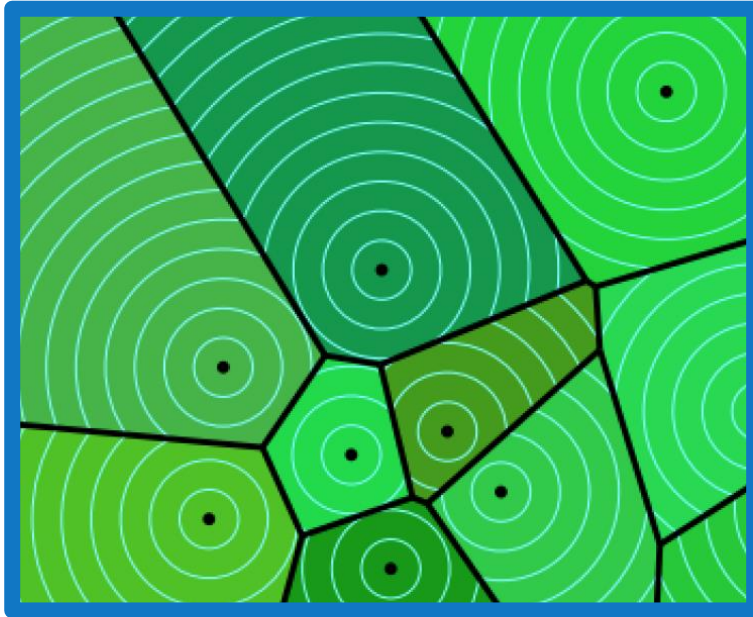
$$\ln(P(A)) - \frac{1}{2}(X - \hat{\mu}_A)^T \hat{\Sigma}_A^{-1} (X - \hat{\mu}_A) :$$
$$= \ln(P(B)) - \frac{1}{2}(X - \hat{\mu}_B)^T \hat{\Sigma}_B^{-1} (X - \hat{\mu}_B)$$

Cannot get rid of quadratic terms, so decision boundary is a quadratic in $\mathbf{X}$.

Summary of GDA Decision Boundaries

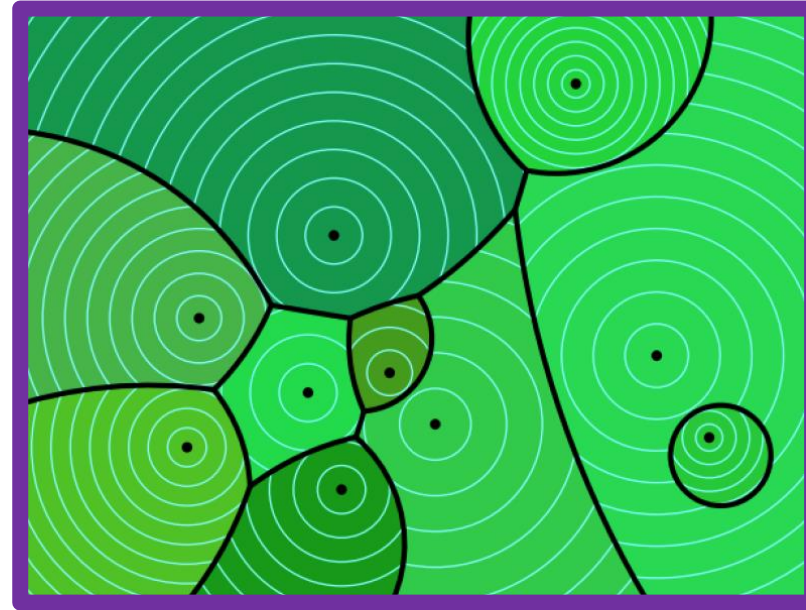
- When class covariances are equal, decision boundary is linear: *Linear Discriminant Analysis* (LDA).
- When class covariances are unequal, decision boundary is quadratic: called *Quadratic Discriminant Analysis* (QDA).
- All these results also hold for multi-class (more than 2 classes).

LDA



$$\Sigma_i = \Sigma_j = \Sigma$$

QDA



$$\Sigma_i \neq \Sigma_j$$

for K classes

Predictive distribution for LDA

- Desired predictive conditional is given by:

$$\begin{aligned} p(Y = a|X) &\propto p(X|Y = a)p(Y = a) \\ &= N(\mu_a, \Sigma)p(Y = a) \\ &\propto \exp[-(X - \mu_a)^T \Sigma^{-1}(X - \mu_a)] \end{aligned}$$

Predictive distribution for LDA

- Desired predictive conditional is given by:

$$\begin{aligned} p(Y = a|X) &\propto p(X|Y = a)p(Y = a) \\ &= N(\mu_a, \Sigma)p(Y = a) \\ &\propto \exp[-(X - \mu_a)^T \Sigma^{-1}(X - \mu_a)] \end{aligned}$$

- Similarly for $p(Y = b|X)$.
- If normalize so that $p(Y = a|X) + p(Y = b|X) = 1$, then

$$p(Y = a|X) = \frac{1}{1 + \exp(-w^T X)}$$

Predictive distribution for LDA

- Desired predictive conditional is given by:

$$\begin{aligned} p(Y = a|X) &\propto p(X|Y = a)p(Y = a) \\ &= N(\mu_a, \Sigma)p(Y = a) \\ &\propto \exp[-(X - \mu_a)^T \Sigma^{-1}(X - \mu_a)] \end{aligned}$$

- Similarly for $p(Y = b|X)$.
- If normalize so that $p(Y = a|X) + p(Y = b|X) = 1$, then

$$p(Y = a|X) = \frac{1}{1 + \exp(-w^T X)}$$

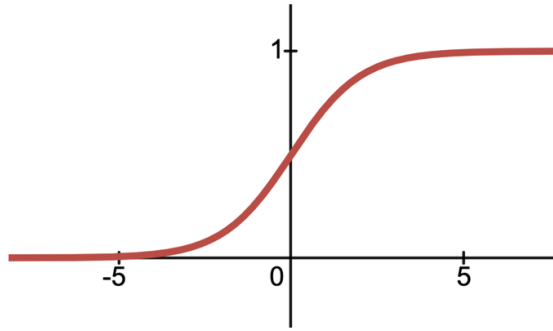
- Training this model directly is the discriminative classifier called *logistic regression*!

Predictive distribution for LDA

The sigmoid squashes $w^T x_i$ to $[0,1]$

- Desirec

$p(Y$



$$\sigma(a) = \frac{1}{1 + e^{-a}}$$

sigmoid function

- Similarl

- If norm

$$p(Y = a|X) = \frac{1}{1 + \exp(-w^T X)}$$

- Training this model directly is the discriminative classifier called *logistic regression*!

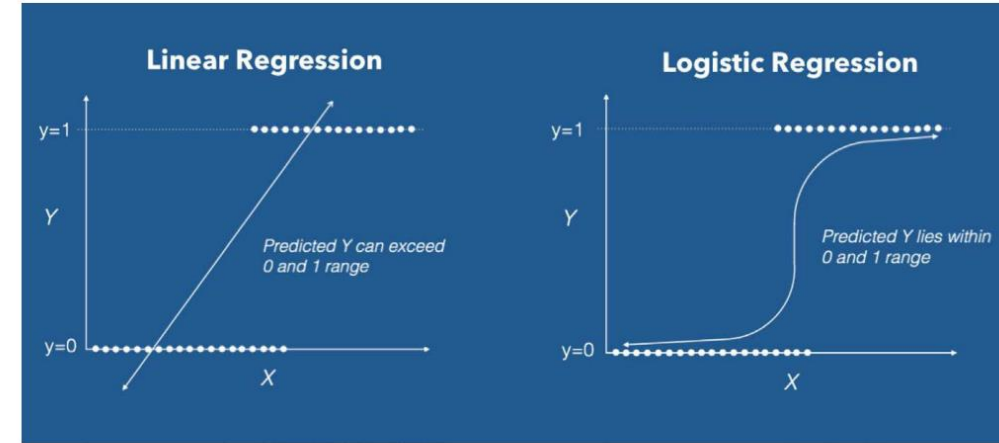
nal is given k

: $a)p(Y = a$

$a, \Sigma)p(Y = ($

$(X - \mu_a)^T \Sigma$

$a|X) + p(Y = a|X) = 1$, then



<https://www.machinelearningplus.com/logistic-regression-tutorial-examples-r/>

LDA vs Logistic regression

1. LDA model can always be written as a logistic regression (LR) model.
2. However, the converse is not true. Not every logistic regression model can be written as an LDA model.
3. Thus LDA makes strictly stronger modelling assumptions than LR, and LR is a more general model.

What about when features, x_i are discrete?

- The “density” part of the generative model, $p(X|Y = a)$ becomes a discrete probability distribution.
- If we assume that all of the features are independent, $p(X|Y = a) = \prod_{f=1}^D p(X_f|Y = a)$ then we get the widely known “Naïve Bayes” model.
- Naïve because of the assumption on independence of features.

Kinds of Classification Problems

Binary classification

- Two classes
- **Responses** (y) are either 0 or 1

win or lose

disease or
no disease

• Multiclass classification

- Many classes
- Image labeling and sentiment analysis (positive, negative, or neutral).



• Structured prediction tasks

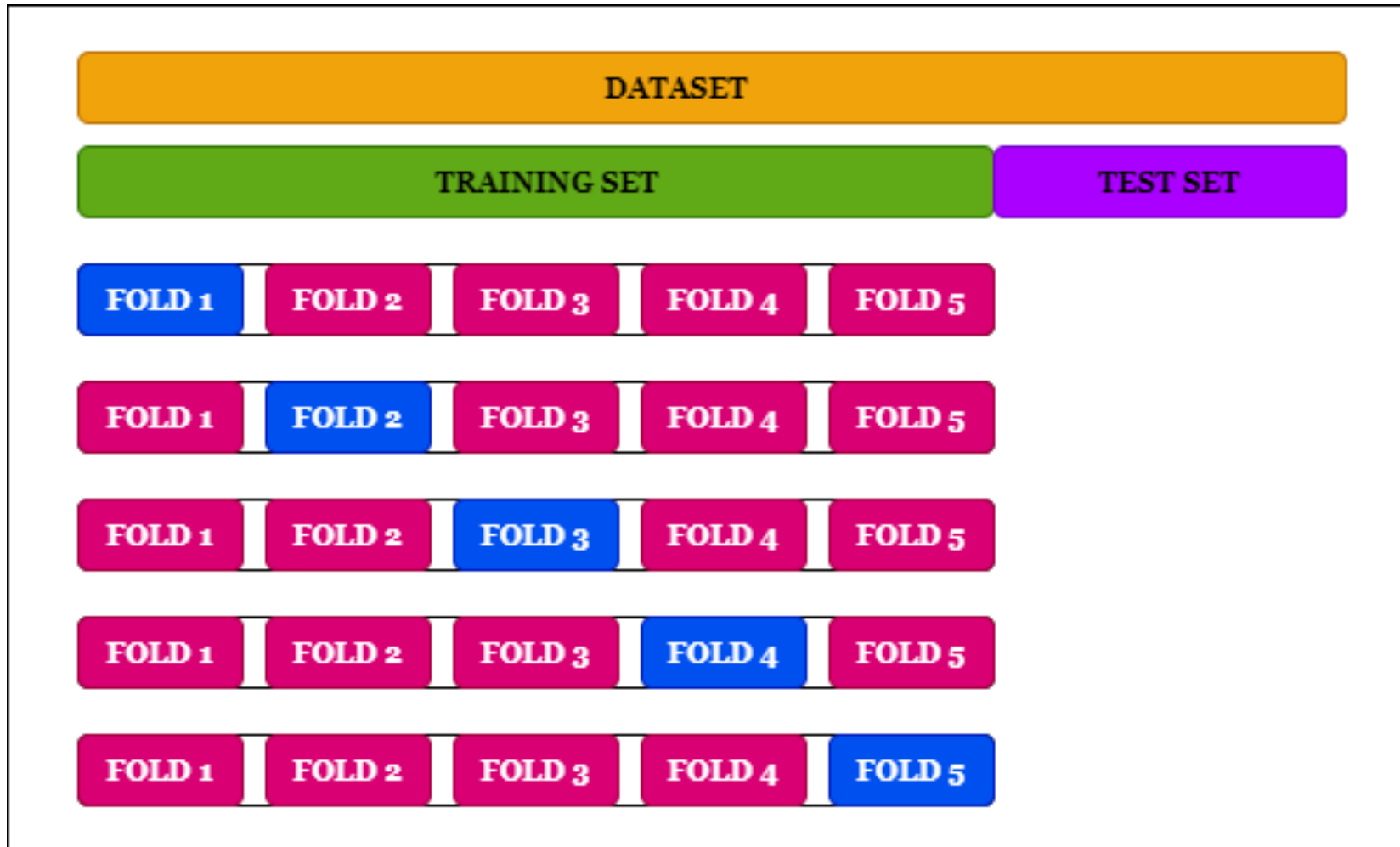
- Predicting a structured object instead of a discrete class
- Examples: Translation and ChatGPT.

CS 189/289

Today's lecture outline:

1. Discriminative vs. Generative Classification
2. Gaussian "Discriminant" Analysis
3. Evaluating Classifiers

Previously: cross-validation (CV)



- CV: re-use your data to **train/validate**, with one final **test set**.
- Suppose we were evaluating classifiers, what might we compute for each test fold?

How to pick between these two classifiers?

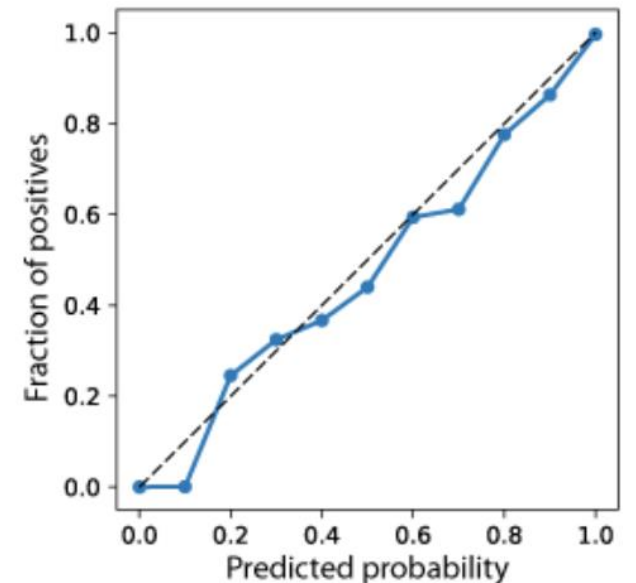
TRUTH	Logistic regression	Neural network
1	0.7198	0.9038
0	0.2460	0.8455
0	0.1219	0.4655
0	0.1560	0.3204
0	0.7527	0.2491
1	0.3064	0.7129
0	0.7194	0.4983
0	0.5531	0.6513
1	0.2173	0.3806
0	0.0839	0.1619
1	0.8429	0.7028

What evaluation metric/quantity applied to these data could help decide which model to use?

How to pick between these two classifiers?

- We could use threshold of $p=0.5$ and count the # of misclassifications that each model makes ("accuracy").
- Assumes probabilities are "calibrated".
- Similarly so does hold out log likelihood.
- Suppose model is not *calibrated*, but there exists a threshold other than 0.5 that yields perfect prediction. Is this a good classifier?
- Also, what if the model is not probabilistic?
- ROC curves are going to help us deal with these issues.

TRUTH	Logistic regression	Neural network
1	0.7198	0.9038
0	0.2460	0.8455
0	0.1219	0.4655
0	0.1560	0.3204
0	0.7527	0.2491
1	0.3064	0.7129
0	0.7194	0.4983
0	0.5531	0.6513
1	0.2173	0.3806
0	0.0839	0.1619
1	0.8429	0.7028



A miscalibrated but useful classifier

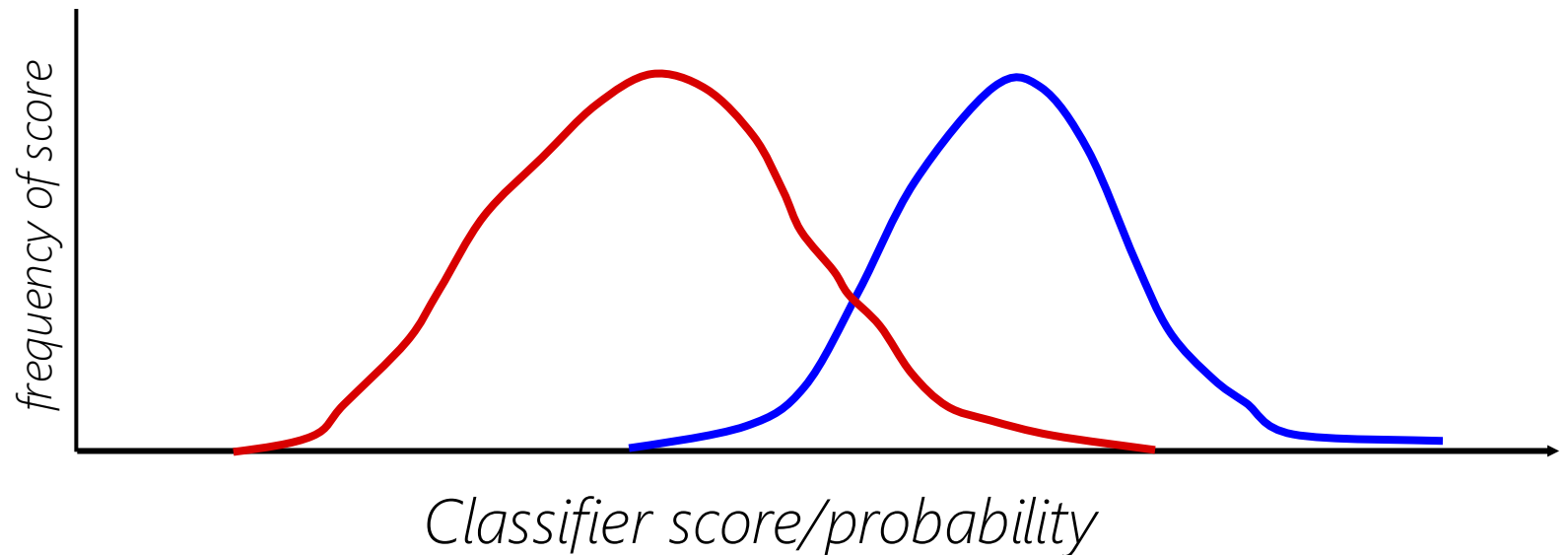
TRUTH	Logistic	Neural network
1	0.88	0.9038
0	0.77	0.8455
0	0.59	0.4655
0	0.81	0.3204
0	0.7527	0.2491
1	0.93	0.63
0	0.7194	0.4983
0	0.5531	0.6513
1	0.98	0.3806
0	0.0839	0.1619
1	0.8429	0.7028

- If we pick the “optimal” decision of $p=0.5$ decision boundary, we will choose the model “containing less information” (NN)!
- Whereas best LR threshold of 0.8 gives 0 errors, while best NN threshold of 0.5 gives 3 errors.

Defining false positives, false negatives, etc.

[We will consider only binary classifiers in today's lecture]

*Distribution of classifier "scores" of
unhealthy and healthy individuals
in a test set*

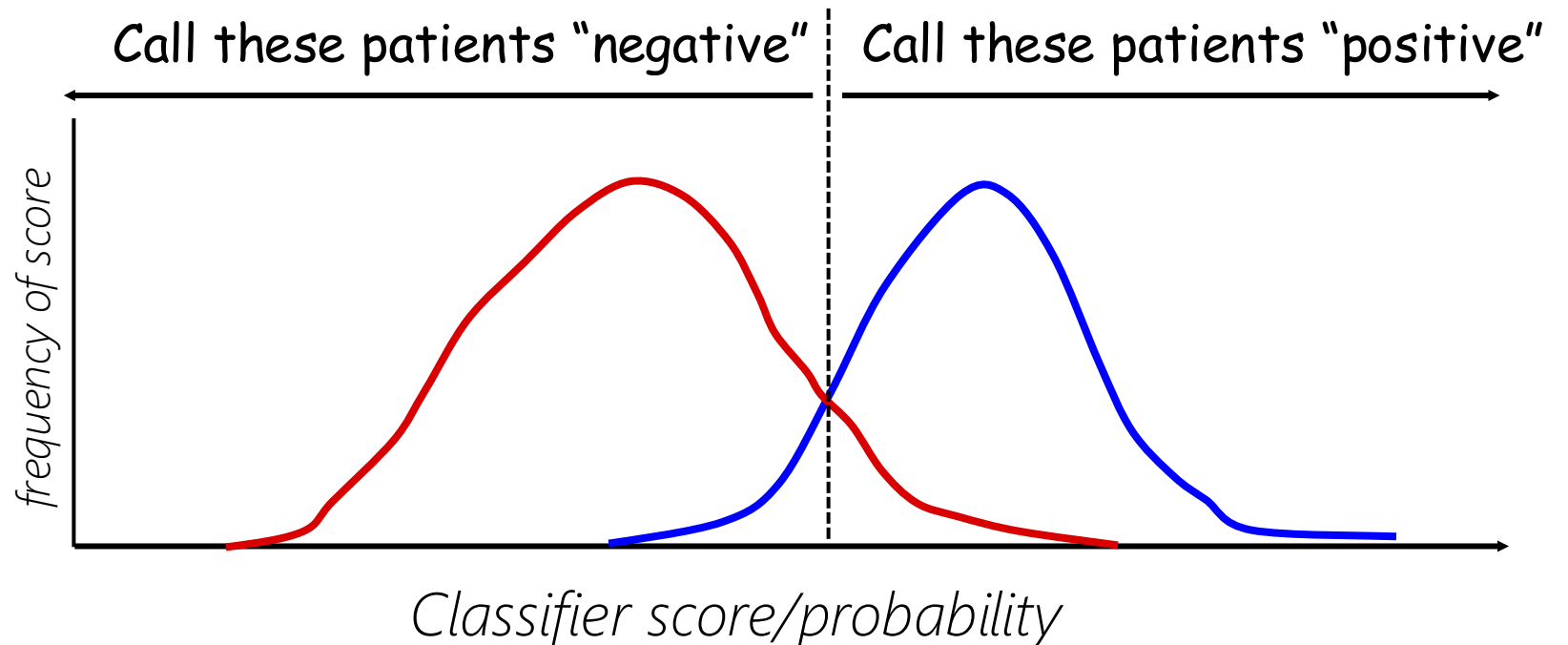


Defining false positives, false negative, etc.

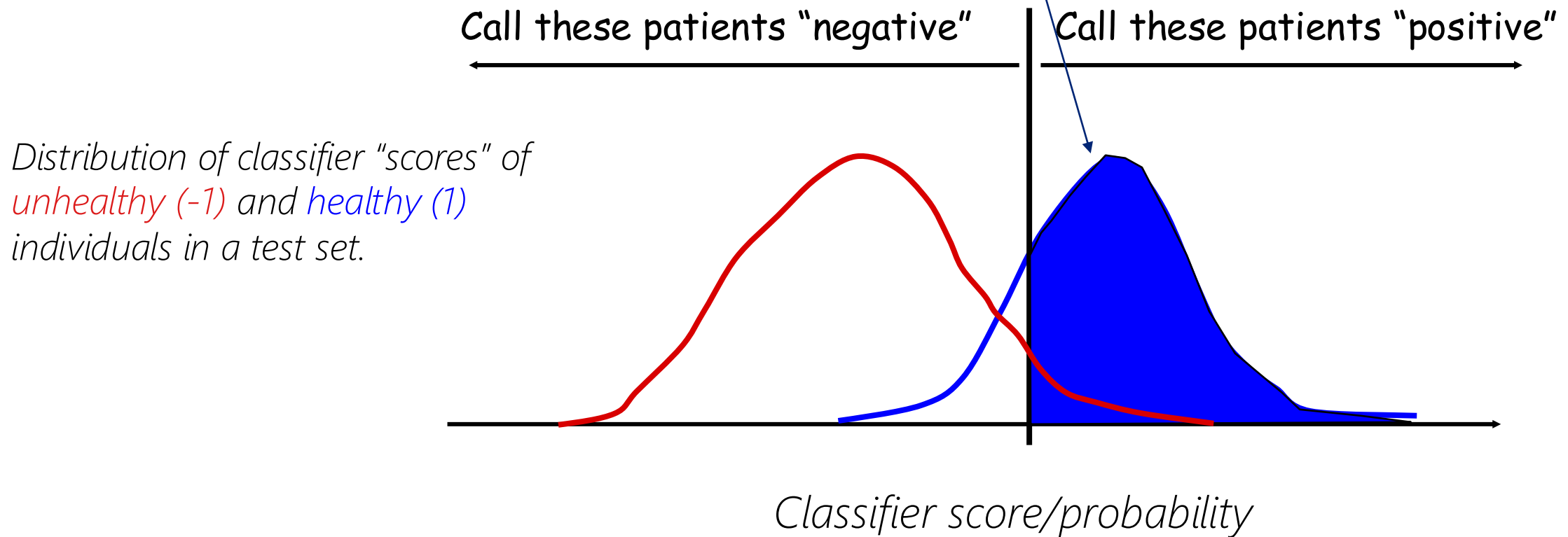
[We will consider only binary classifiers in today's lecture]

Choose a threshold on the score/probabilistic output, and call/predict all samples above it a "1" (e.g. "healthy") and all those below it a "-1" (e.g. "unhealthy").

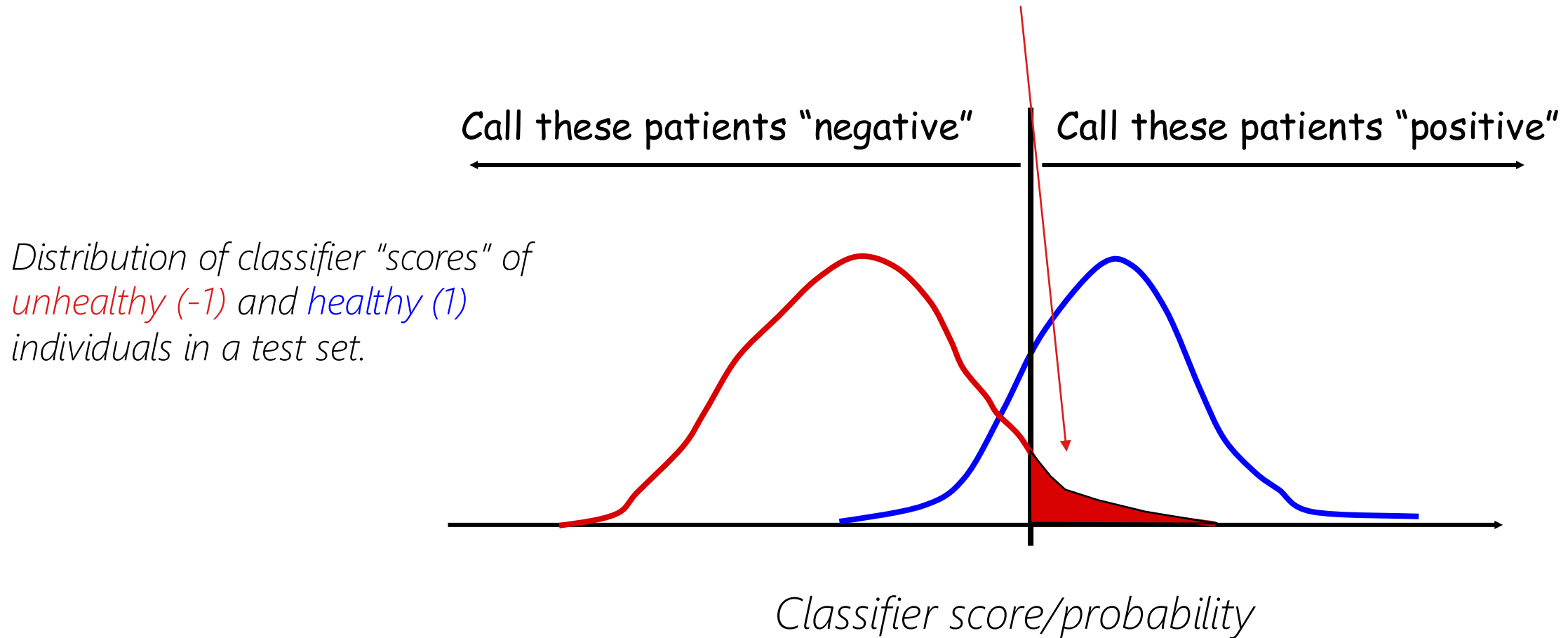
*Distribution of classifier "scores" of
unhealthy and healthy individuals
in a test set*



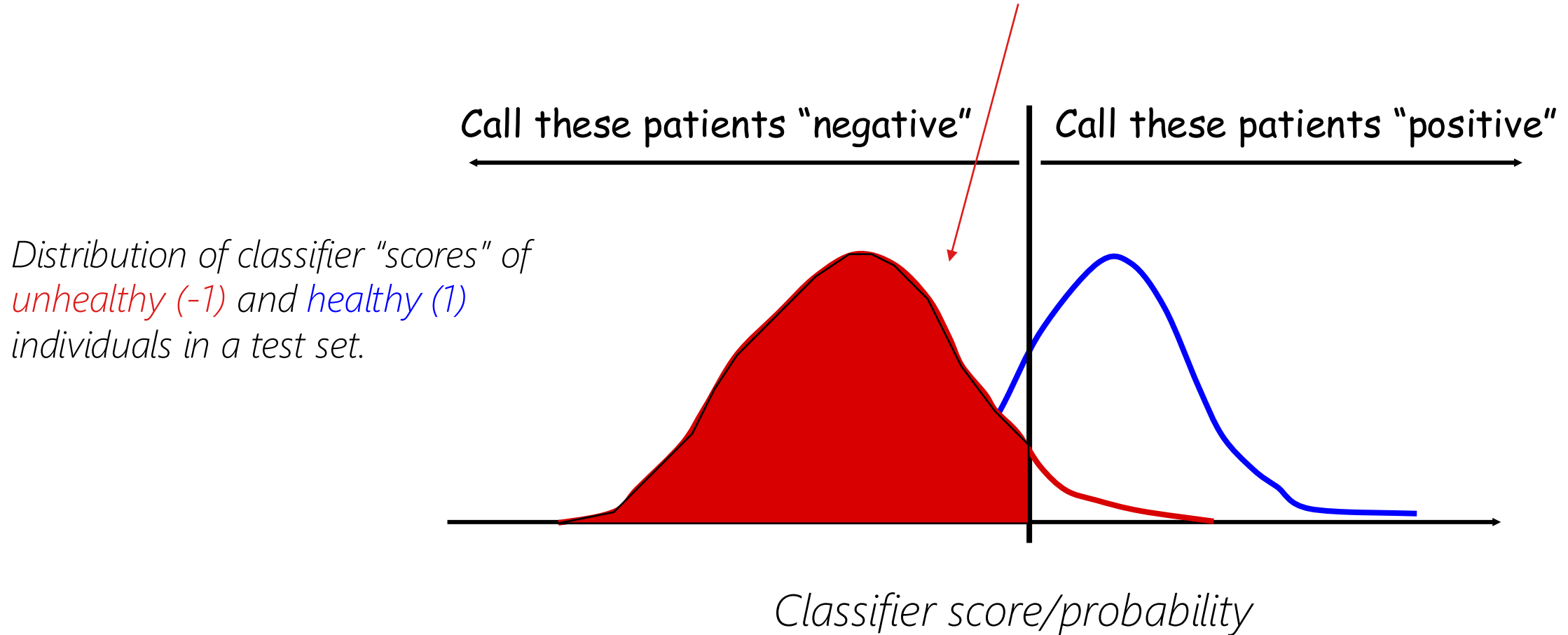
Definitions: True Positives (TP)



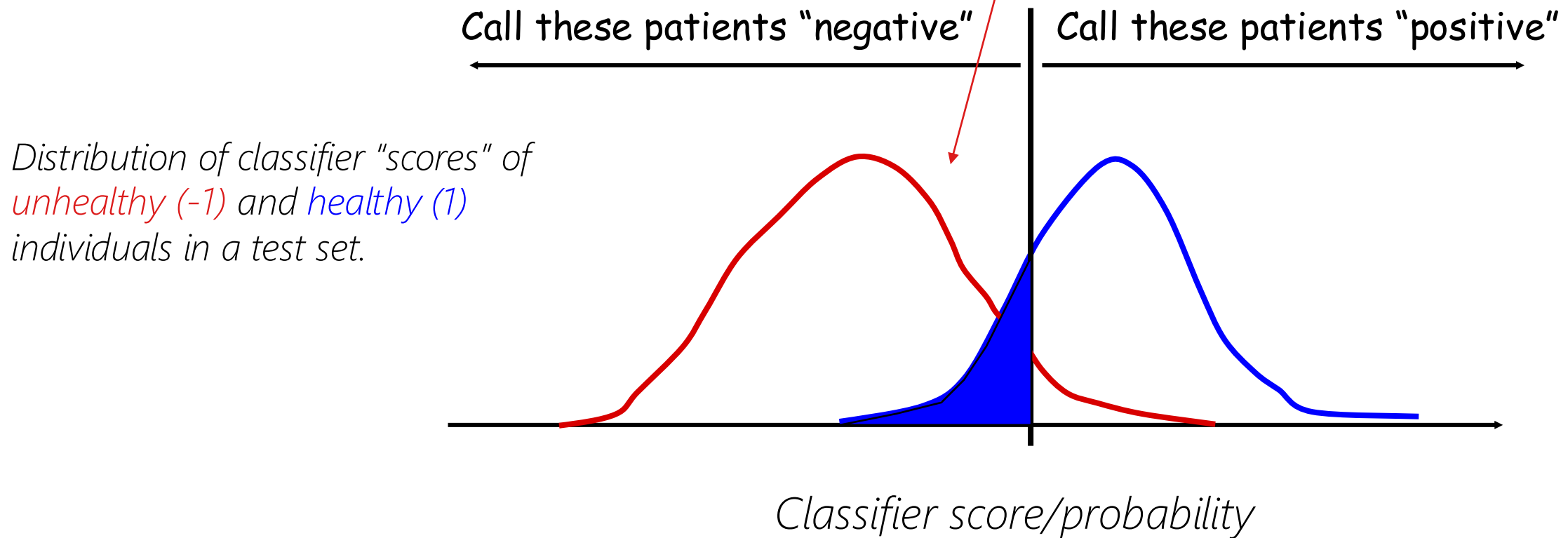
Definitions: False Positives (FP)

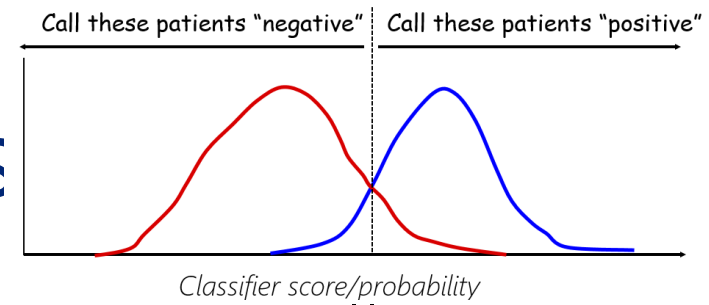


Definitions: True Negatives (TN)



Definitions: False Negatives (FN)





Summary of classifier decision outcomes

Once we set a *decision* threshold on the predictive score, all test data points fall into one of these four categories:

1. False Positive (FP)—person is truly a "-1" but called "1"
 2. False Negative (FN)—person is truly a "1" but called "-1"
 3. True Positive (TP) —person is truly a "1" and called "1"
 4. True Negative (TN) —person is truly a "-1" and called "-1"
- Thus if N is total # of test points, then $N = FP + FN + TP + TN$
 - FP and FN are mistakes when using the classifier.
 - TP and TN are correct decisions when using the classifier.

Summary of classifier decision outcomes

Often we see these reported in the form of a *confusion matrix*:

		MODEL PREDICTIONS		
		<i>Negative</i>	<i>Positive</i>	
GROUND TRUTH	<i>Negative</i>	TN	FP	#actual negatives = TN + FP
	<i>Positive</i>	FN	TP	#actual positives = FN + TP

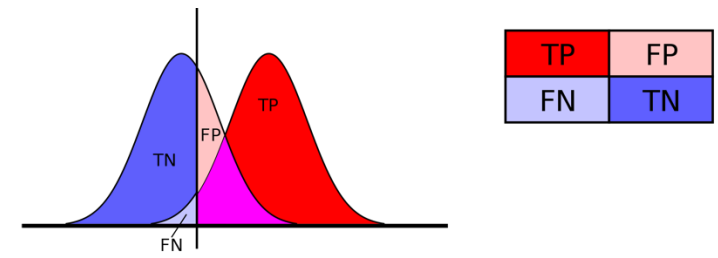
Summary of classifier decision outcomes

Rates: "normalize" by #samples who could have had that call:

- TP rate, $\text{TPR} = \text{TP} / \# \text{actual positives} = \text{TP} / (\text{FN} + \text{TP})$, aka *Sensitivity*
- TN rate, $\text{TNR} = \text{TN} / \# \text{actual negatives} = \text{TN} / (\text{TN} + \text{FP})$, aka *Specificity*
- FN rate, $\text{FNR} = \text{FN} / \# \text{actual positives} = 1 - \text{TPR}$ aka *Miss Rate*
- FP rate, $\text{FPR} = \text{FP} / \# \text{actual negatives} = 1 - \text{TNR}$ aka *Fall out*

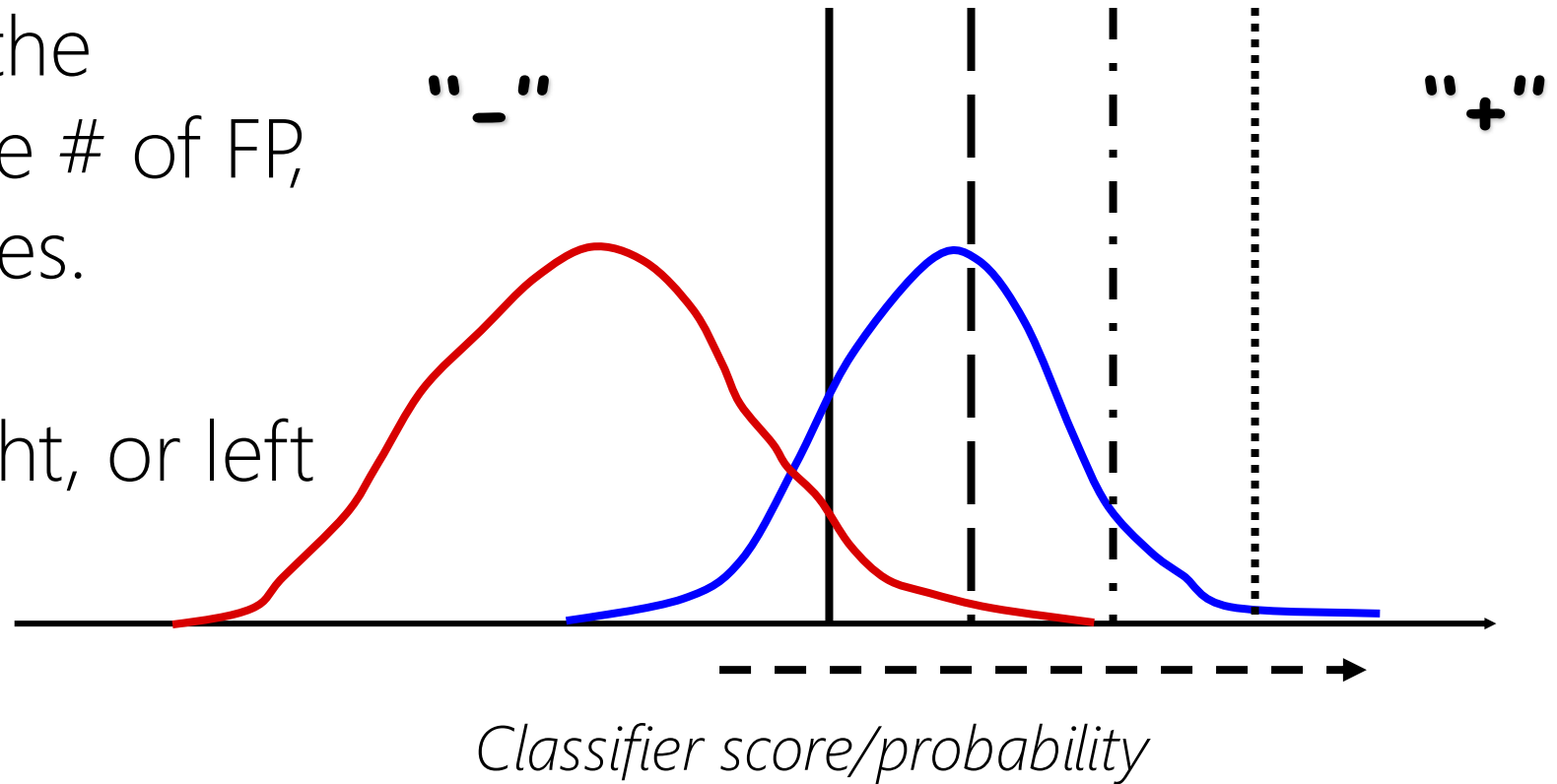
		MODEL PREDICTIONS		
		Negative	Positive	
GROUND TRUTH	Negative	TN	FP	#actual negatives = TN + FP
	Positive	FN	TP	#actual positives = FN + TP

Back to our decision threshold

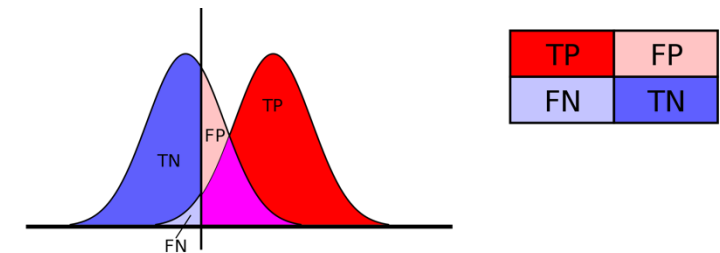
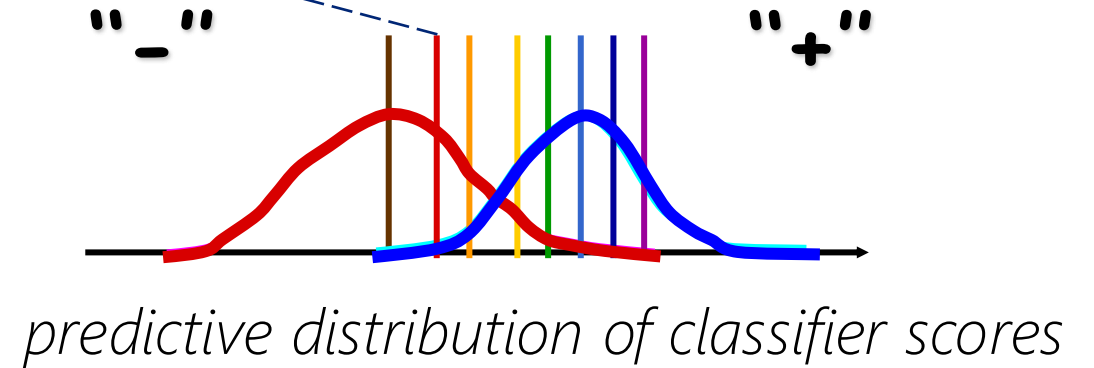
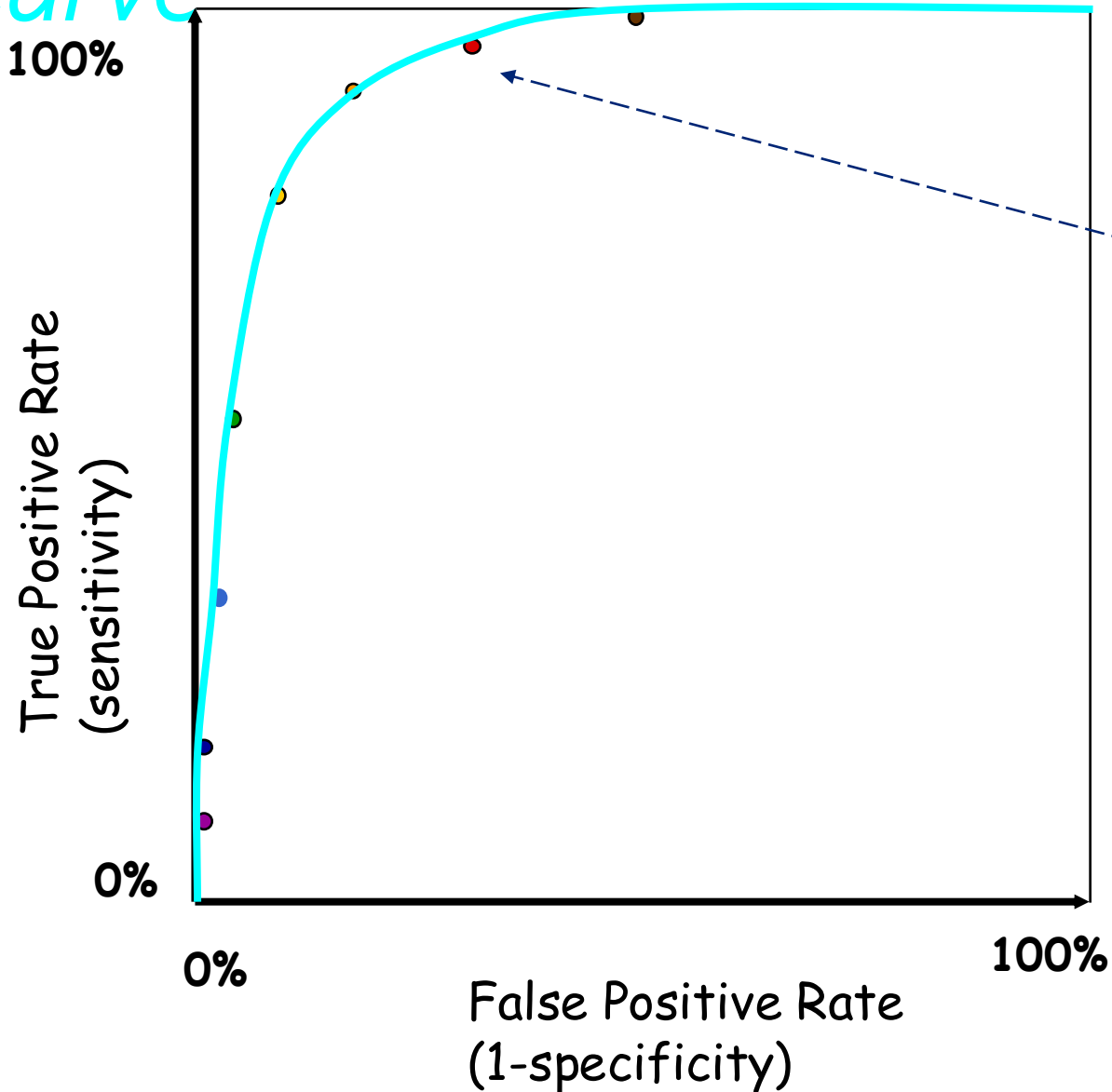


Every time we move the decision threshold, the # of FP, FN, TP and TN changes.

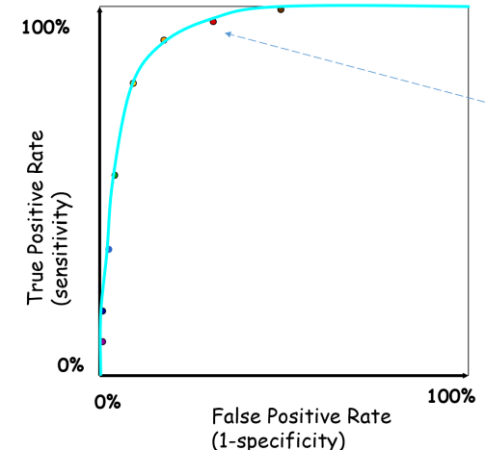
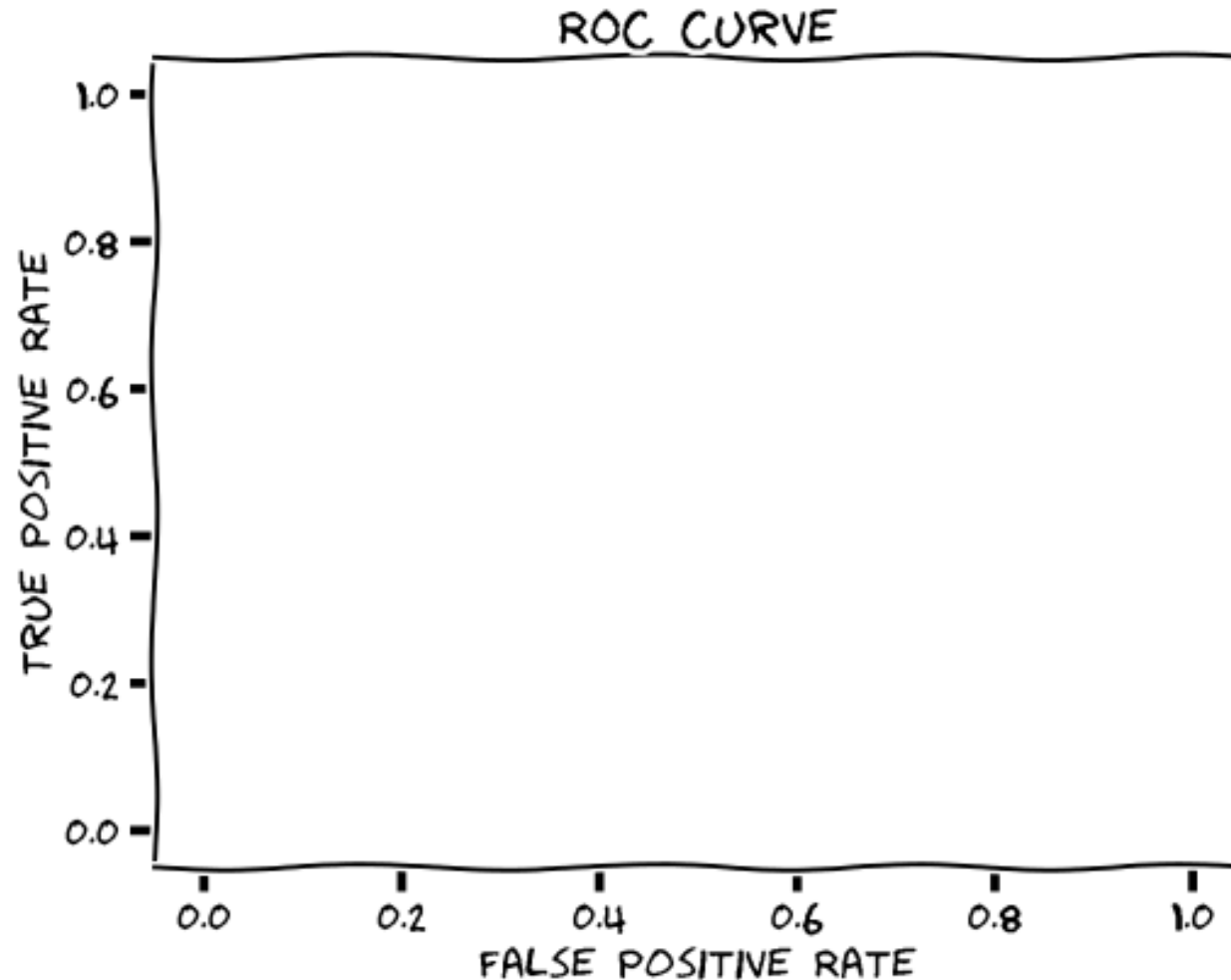
e.g. shifting to the right, or left



As we shift it, we can draw out an *ROC curve*

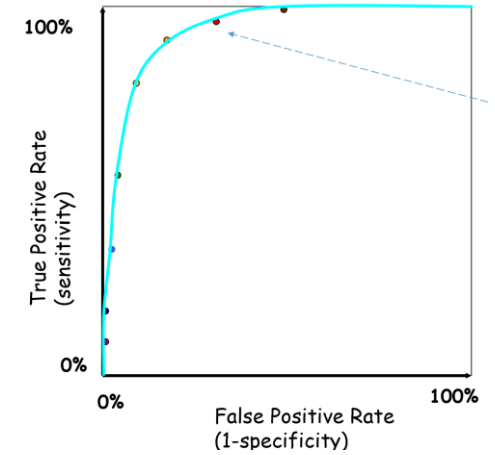
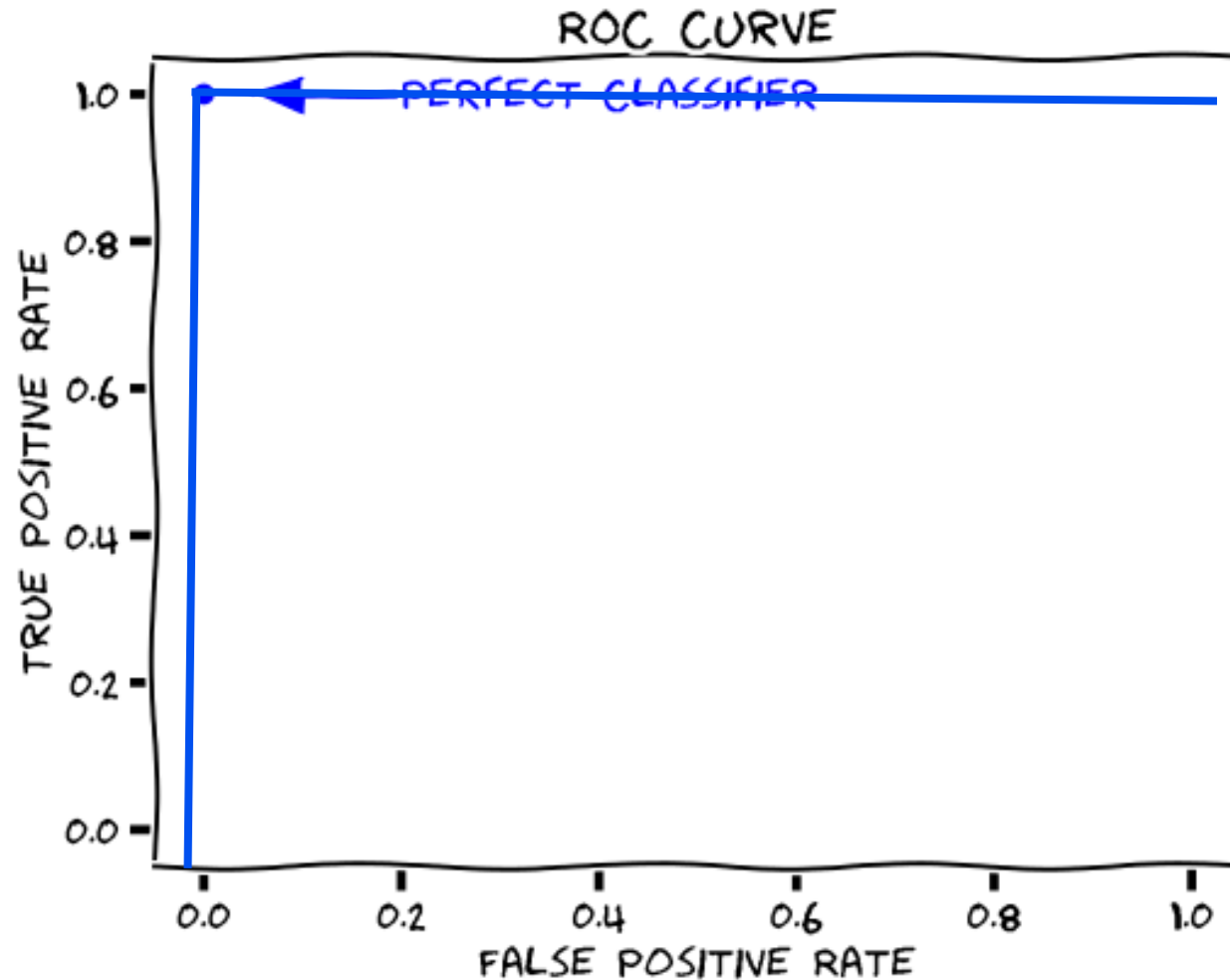


Comparing ROC curves across classifiers

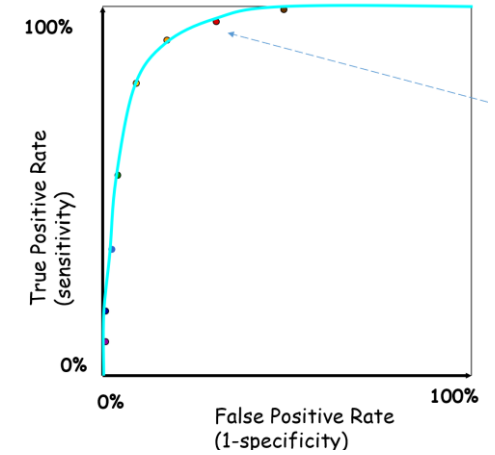
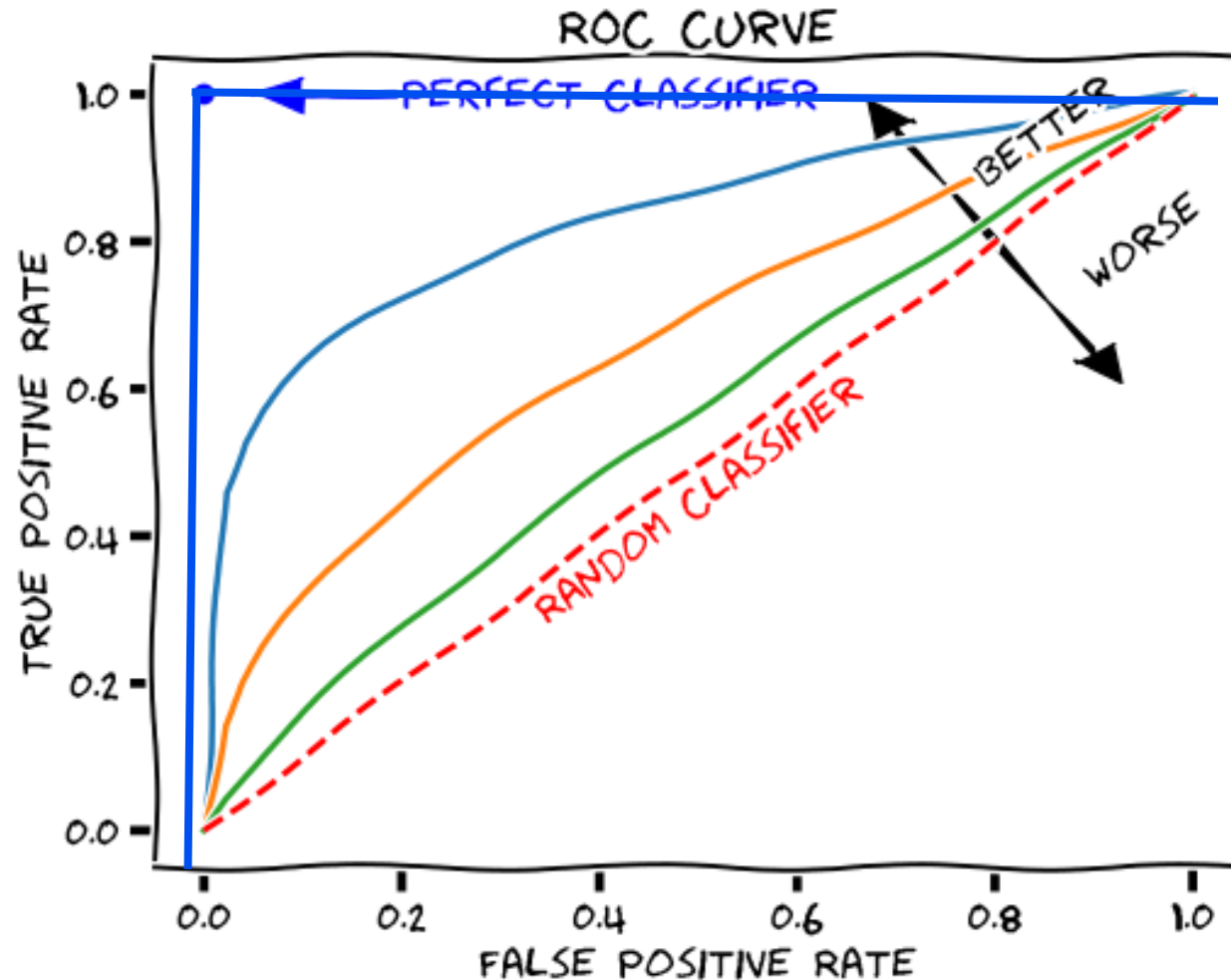


How would a perfect classifier appear on this plot?

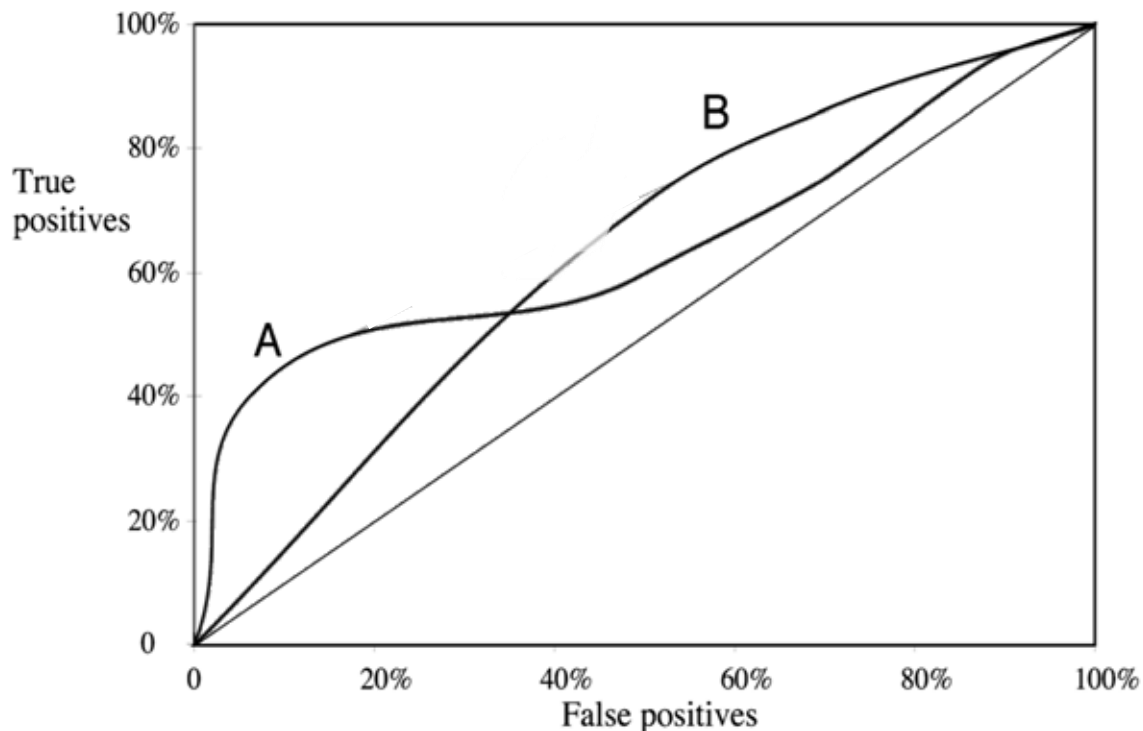
Comparing ROC curves across classifiers



Comparing ROC curves across classifiers



Which model would you choose?



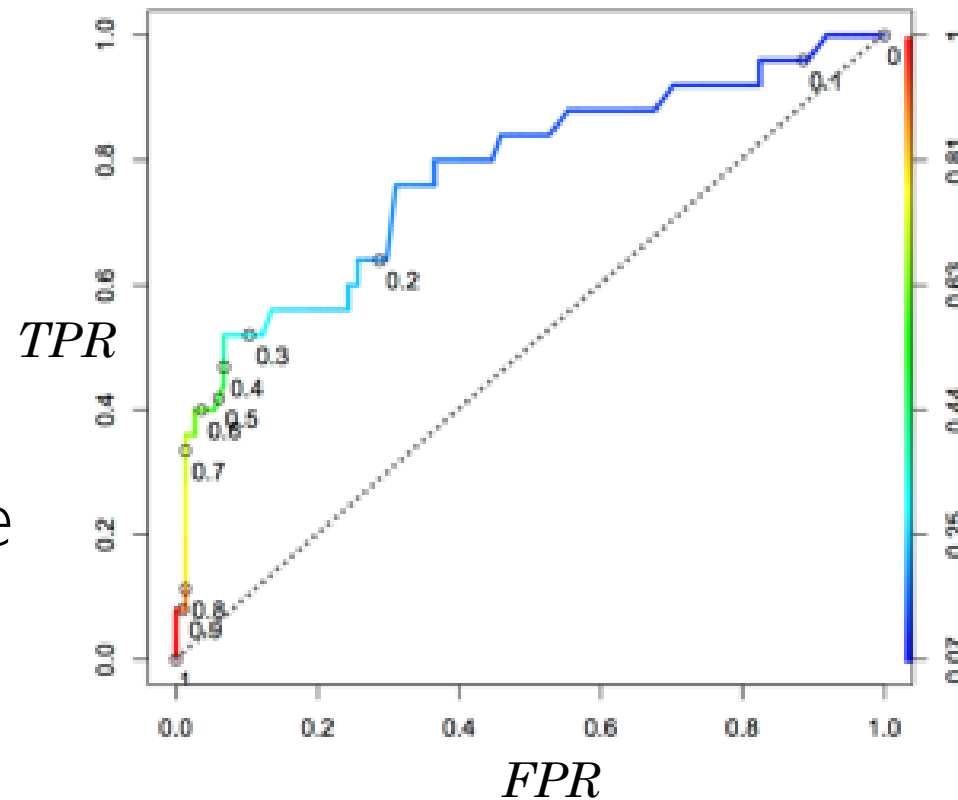
- In this example, no one classifier (A or B) *dominates* in ROC space.
- Thus we might ask what regime is most relevant to our problem.
- *e.g.* suppose you know you need few false positives, then you should pick method A, which dominates up until FPR 40%.

An algorithm for making an ROC curve

Exploit the property that any instance that is classified as + at a given threshold, will be classified as + for all lower thresholds as well:

1. Sort the test instances by decreasing score.
2. Move down the list (lowering the threshold), processing one instance at a time. For each instance,
3. Computer the TPR and FPR and add one point to the plot.

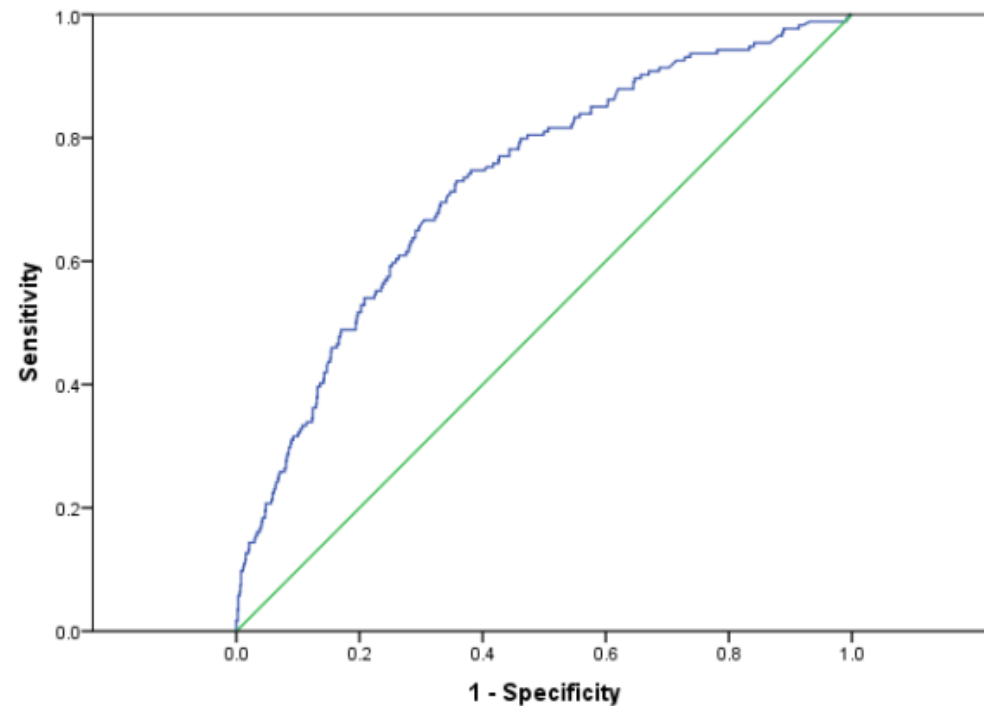
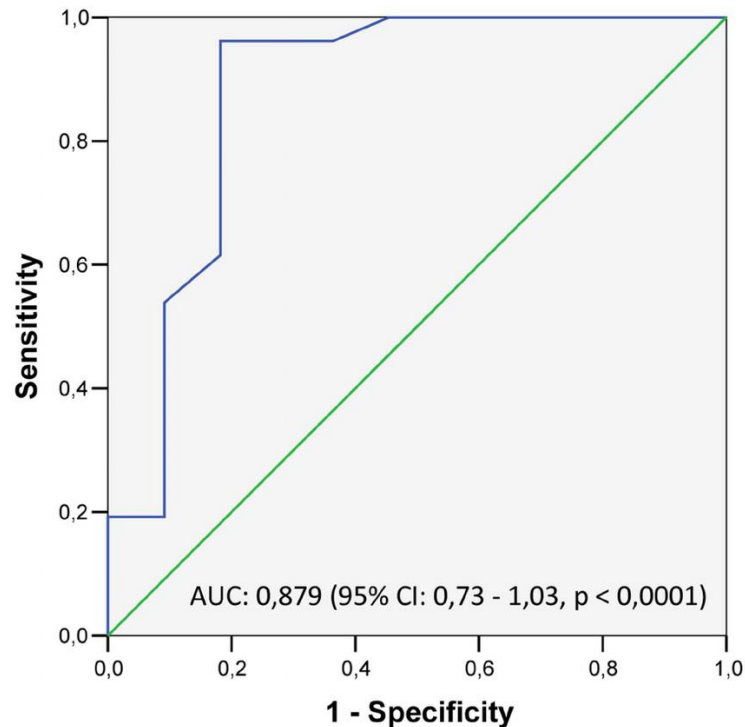
*Table does not correspond to figure



TRUTH	LOGISTIC Regression score
1	0.98
1	0.93
0	0.88
1	0.8429
0	0.81
1	0.77
0	0.7527
0	0.7194
1	0.59
0	0.42
0	0.0839

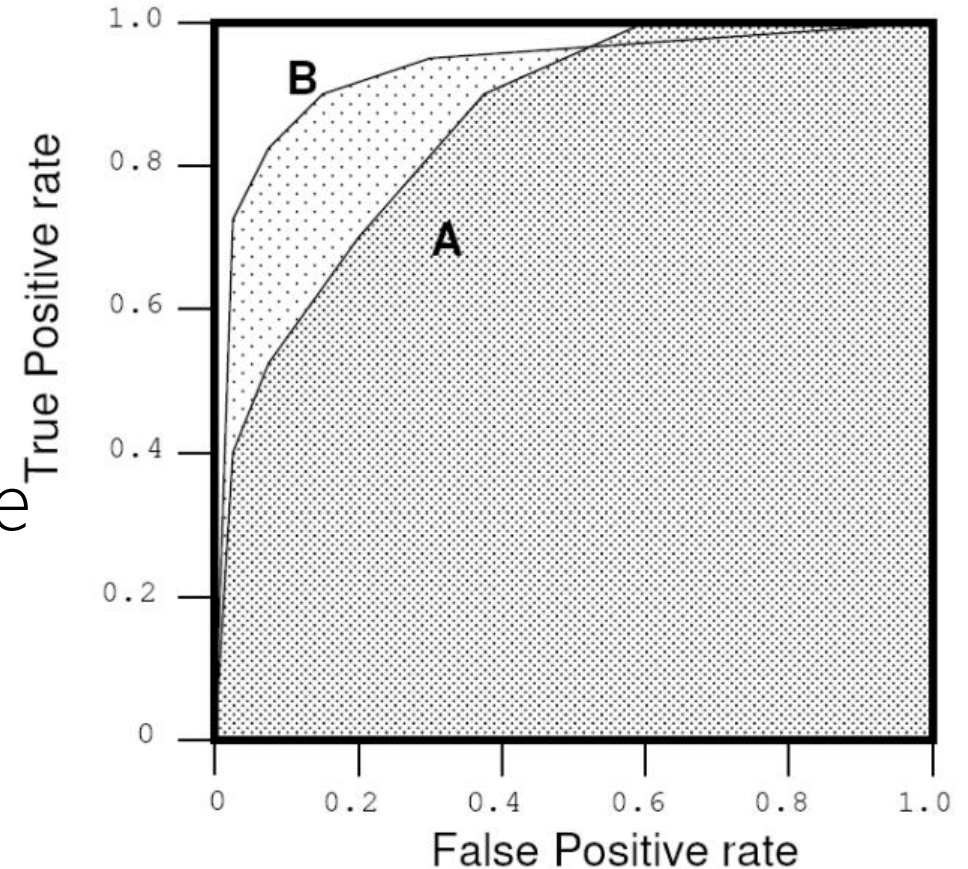
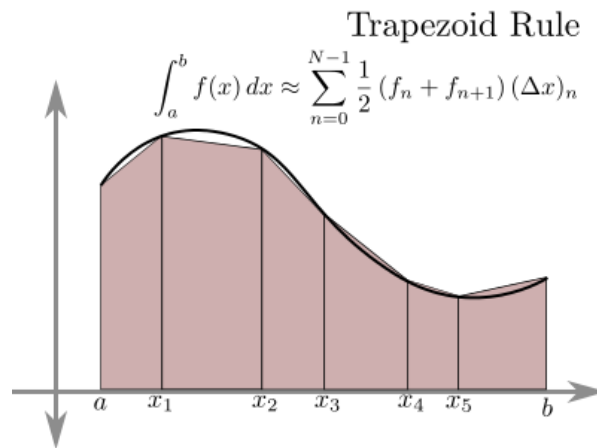
An algorithm for making an ROC curve

- Smoothness of the ROC curve is dependent on the # of points in it
- Restricted by # of test points and uniqueness of scores:

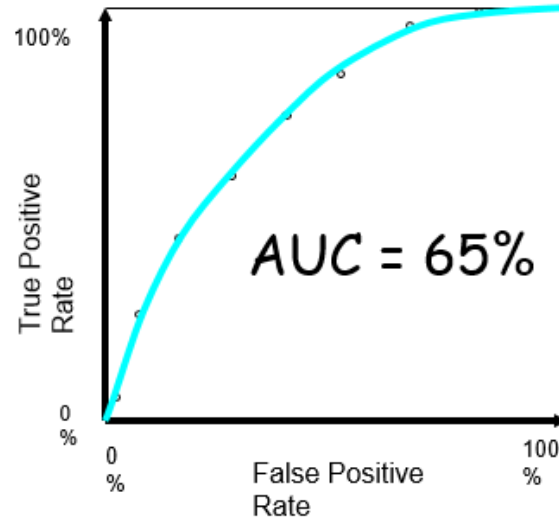
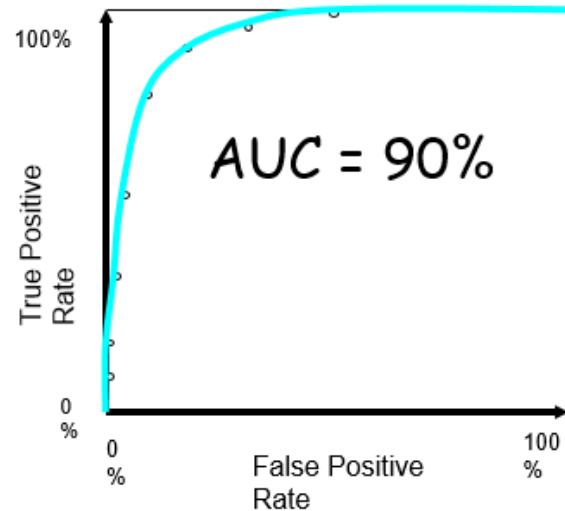
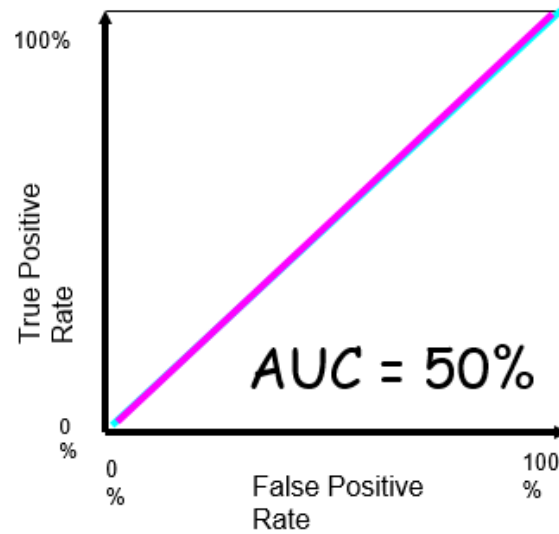
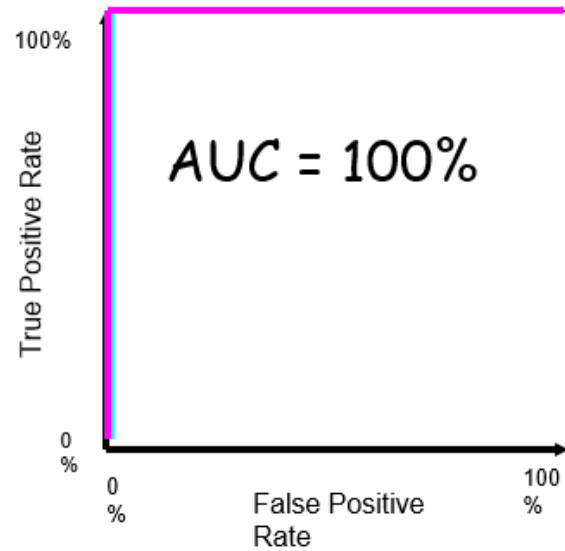


Summarizing ROCs with the Area Under the Curve (AUC)

- AUC: often used to compare classifiers.
- The bigger the AUC the better.
- AUC can be computed by a slight modification to the algorithm for constructing ROC curves—basically a simple form of integration to compute the area under the curve.



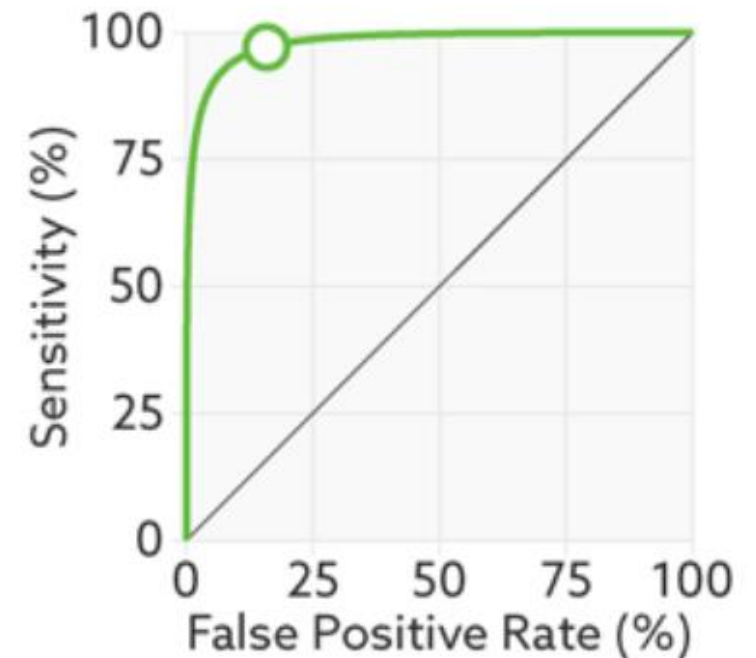
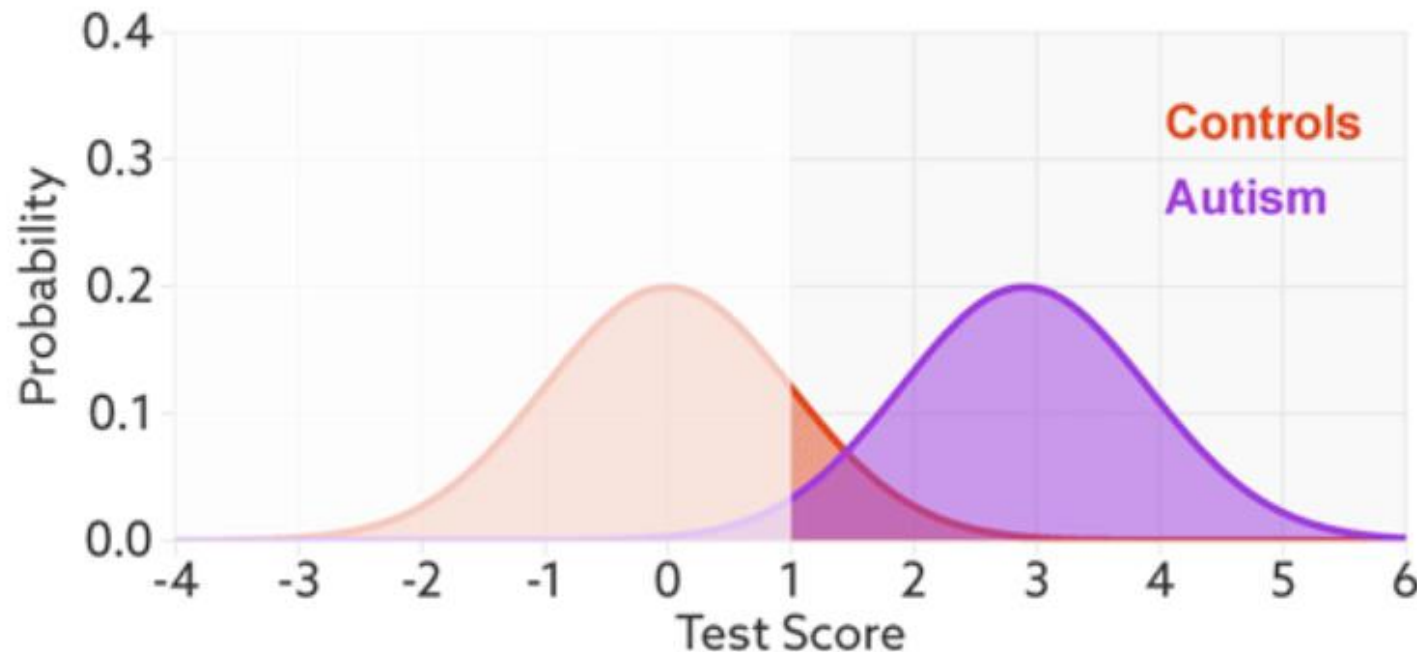
Summarizing ROCs with the Area Under the Curve (AUC)



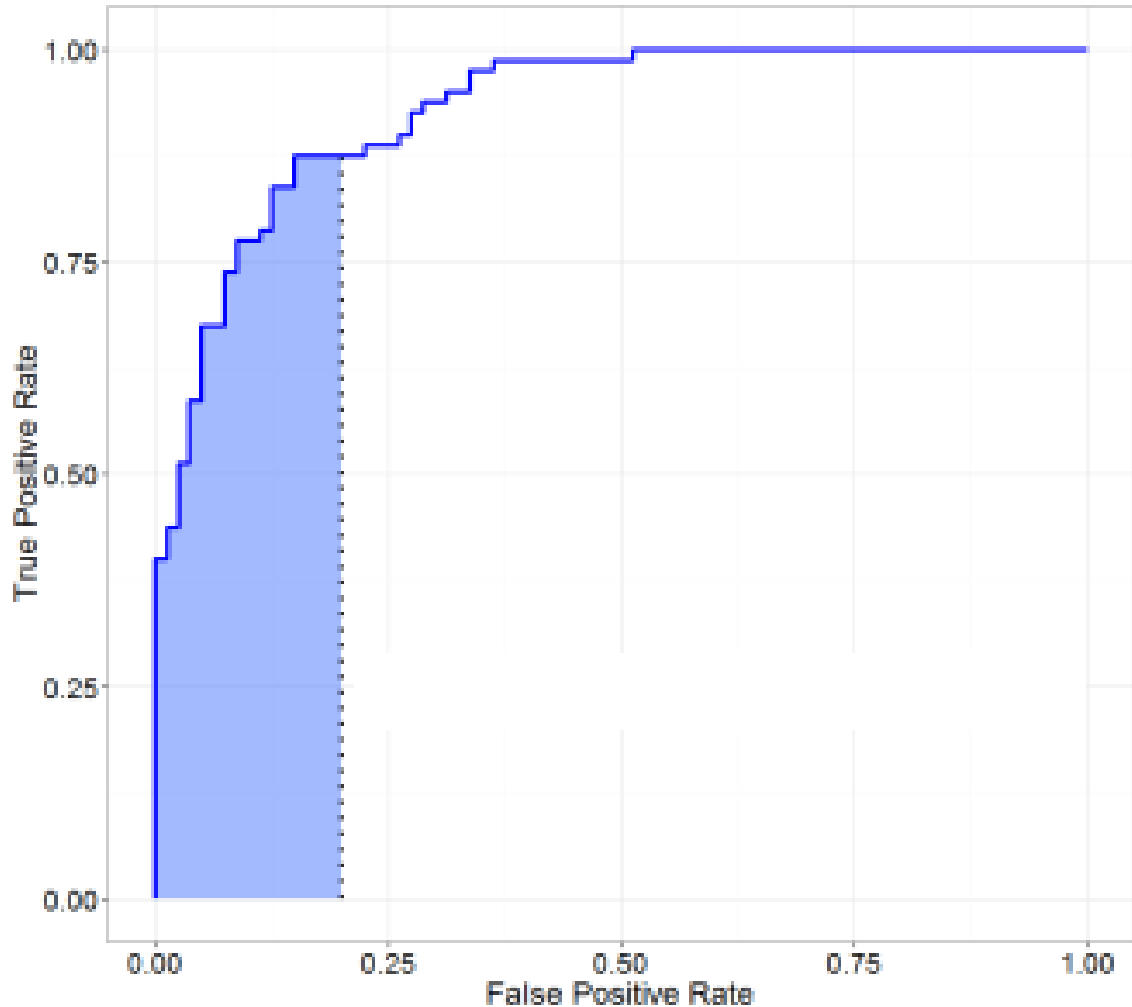
The AUC of a classifier is equivalent to the probability that the classifier will rank a randomly chosen positive sample higher than a randomly chosen negative sample.

Visualization of score distributions wrt ROCs & AUC

1. Sweeping a threshold through the predictions for one classifier traces out the ROC curve.
2. The more separated the distribution of scores between the two classes, the larger the AUC.



Partial Area Under the Curve (AUC)

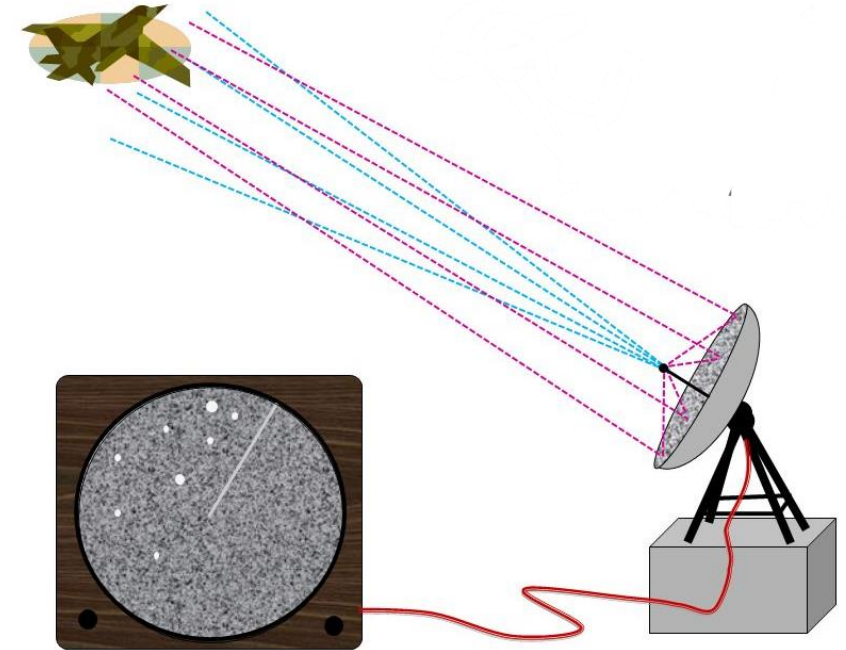


Sometimes we know we would never operate with a FPR above some amount, and so we compute a *partial* AUC, here $AUC(0.2)$.

- e.g. deciding on medical intervention like chemotherapy
- e.g. deciding to spend lots of \$\$\$ in follow up biology experiments.

History of ROC curves

- "The ROC curve was first developed by electrical engineers and radar engineers during World War II (1939-45) for detecting enemy objects in battle fields and was soon introduced to psychology to account for perceptual detection of stimuli."
- "ROC analysis since then has been used in medicine, radiology, biometrics, and other areas for many decades and often used in machine learning and data mining research and applications."



More on ROC curves

ROC curves are **insensitive to the balance of classes** in the test set (because FPR and FNR are insensitive quantities).

- TP rate, $\text{TPR} = \text{TP} / \# \text{actual positives} = \text{TP} / (\text{FN} + \text{TP})$
- TN rate, $\text{TNR} = \text{TN} / \# \text{actual negatives} = \text{TN} / (\text{TN} + \text{FP})$

- To obtain classification accuracy from an ROC, need to know the balance (i.e. ratio of # actual positives to # actual negatives in the test set).
- Knowing this we can find a point on the graph with optimal classification accuracy.
- Sometimes we use Precision-Recall curves instead of ROC because desire sensitivity to the balance of classes.

Summary of classifier decision outcomes

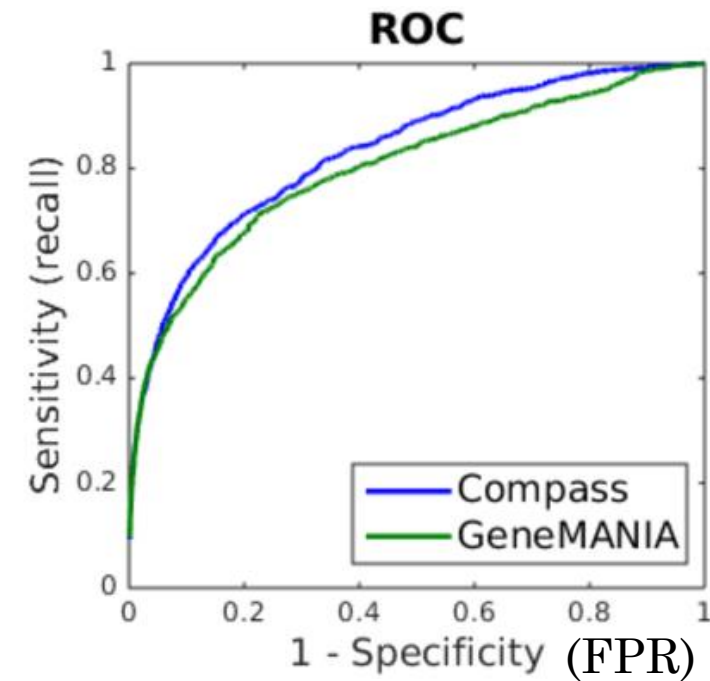
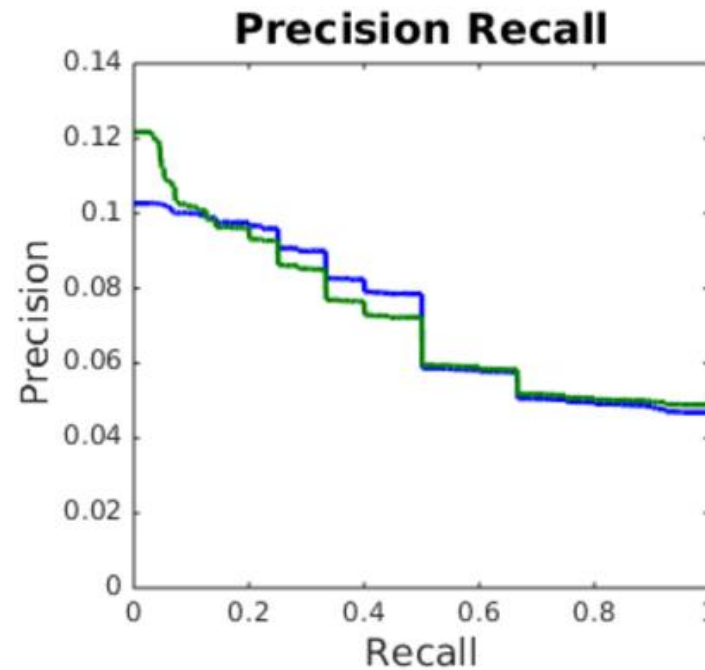
Rates: “normalize” by #samples who could have had that call:

- TP rate, $\text{TPR} = \text{TP} / \# \text{actual positives} = \text{TP} / (\text{FN} + \text{TP})$, aka “Sensitivity”
- TN rate, $\text{TNR} = \text{TN} / \# \text{actual negatives} = \text{TN} / (\text{TN} + \text{FP})$, aka “Specificity”
- FN rate, $\text{FNR} = \text{FN} / \# \text{actual positives} = 1 - \text{TPR}$ aka “Miss Rate”
- FP rate, $\text{FPR} = \text{FP} / \# \text{actual negatives} = 1 - \text{TNR}$ aka “Fall out”
- Precision = $\text{TP} / (\# \text{predicted positive}) = \text{TP} / (\text{TP} + \text{FP})$ —this now **depends on class balance** in test set.
- Recall = $\text{TPR} = \text{Sensitivity}$

		MODEL PREDICTIONS		
		Negative	Positive	
GROUND TRUTH	Negative	TN	FP	#actual negatives = TN + FP
	Positive	FN	TP	#actual positives = FN + TP

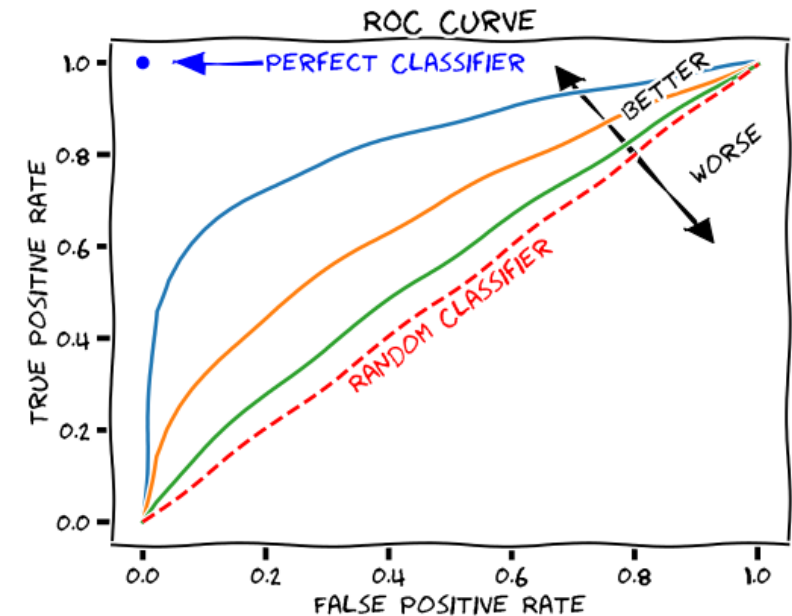
Precision Recall Curves

- Wrt ROC: replace FPR with Precision and flip the axes.
- ROC curves useful when we want invariance to class distribution.
- Precision-recall curves useful when care about (and know) the balance of the classes at test time.



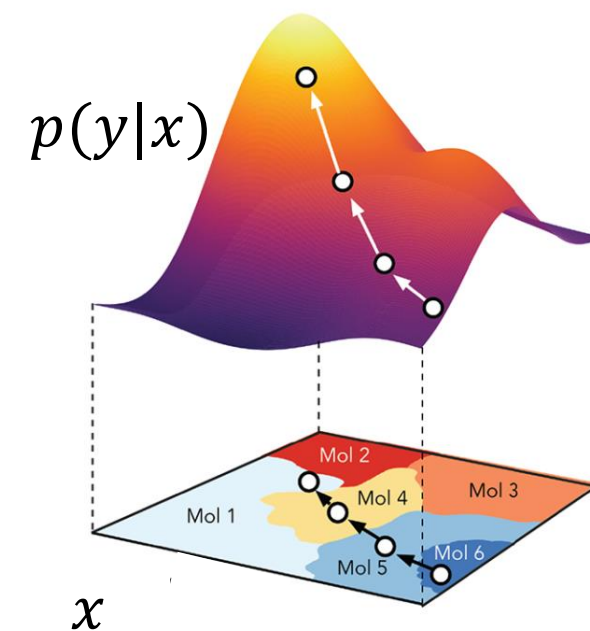
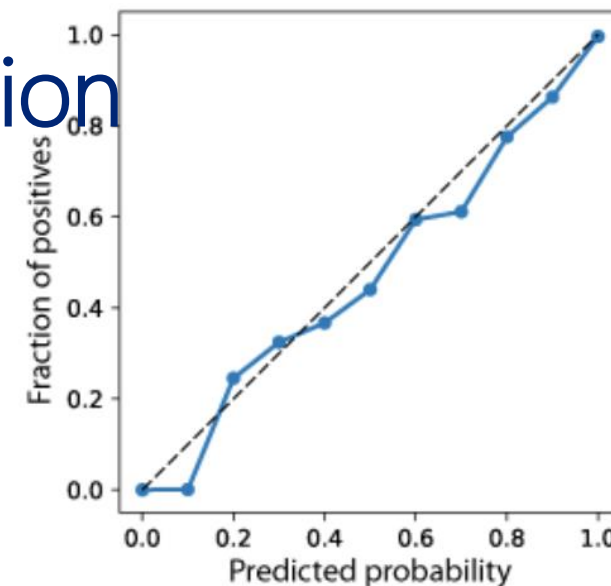
Summary of ROC curves and their utility

- Gives more nuanced understanding than counting the # of misclassifications.
- Does not require a decision threshold.
- Summarizes performance of binary classification models, across all possible trade-offs in decision making FPR and FNR.
- Does not care about model calibration (can be a pro or a con).
- Can compare classifiers by comparing their AUC summary, or using the entire ROC curves, or just part of curve for AUC/ROC.



If you care about (probabilistic) model calibration

- Then you should NOT use ROC curves to evaluate, because they don't care about "calibrated uncertainty" of the predictions.
- Calibrated uncertainty can be very important in medical applications, such as to determine treatments, or administering invasive diagnostics.
- Calibration also come into ML-based design---e.g. in small molecule engineering (e.g. use a predictive model to design the best binder to drug target).
- Also in "active learning".



Many other evaluations—dictated by the domain of application

- Predict a Ranking (of webpages)

- Users only look at top 4
- Sort by $f(x|w,b)$

- Precision @4 = 1/2
 - Fraction of top 4 relevant

- Recall @4 = 2/3
 - Fraction of relevant in top 4

- Top of Ranking Only!

Top 4

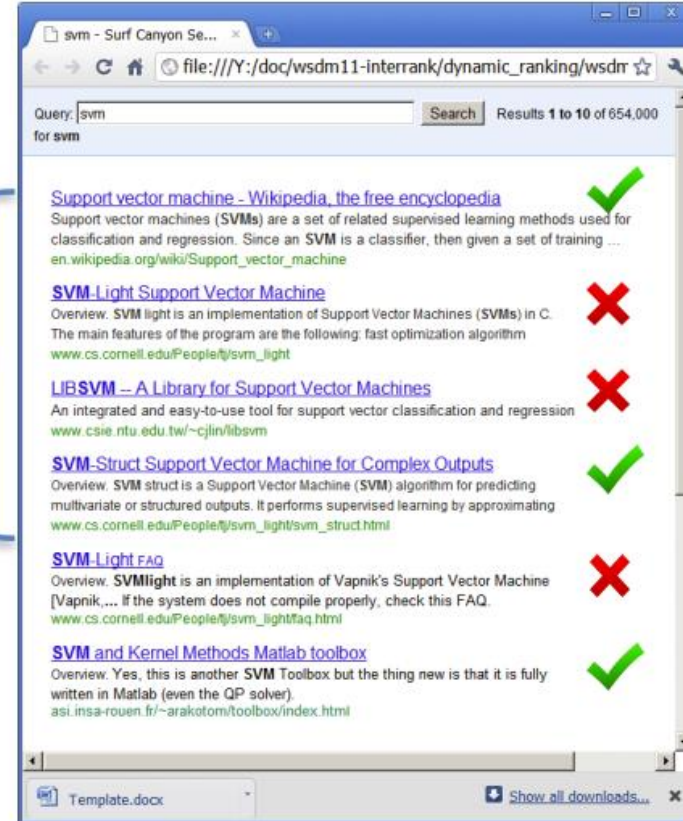


Image Source: http://pmtk3.googlecode.com/svn-history/r785/trunk/docs/demos/Decision_theory/PRhand.html

If we care about these metrics, why not use them as a loss function to train the model?

- These losses are often not differentiable everywhere (e.g. AUC and others have hard thresholding).
- Can't use mini-batch (ranking requires entire test set), hence can't use SGD.
- Some niche attempts to make metrics differentiable, but in practice, rarely used.

Learning cost-sensitive classifiers

- In regular classification (all we have seen so far), we treat all types of misclassification errors the same (e.g. in using a likelihood loss/cost function).
- A more general setting is to learn *cost-sensitive* classifiers where we allow for different types of errors to have more or less impact.

mpact.

Model	Patient		
	Loss Function	Has Cancer	Doesn't Have Cancer
	Predicts Cancer	Low	Medium
	Predicts No Cancer	OMG Panic!	Low

Learning cost-sensitive classifiers

- In regular classification (all we have seen so far), we treat all types of misclassification errors the same (e.g. in using a likelihood loss/cost function).
- A more general setting is to learn *cost-sensitive* classifiers where we allow for different types of errors to have more or less impact.

mpact.

Model	Patient		
	Loss Function	Has Cancer	Doesn't Have Cancer
	Predicts Cancer	Low	Medium
	Predicts No Cancer	OMG Panic!	Low

Optimizing for Cost-Sensitive Loss

- No universally accepted way, but a common and simple one is to use a “Cost Balancing” loss (effectively you are “rebalancing” the training data by your costs):

$$\operatorname{argmin}_{w,b} \left(1000 \sum_{i:y_i=1} L(y_i, f(x_i | w, b)) + \sum_{i:y_i=-1} L(y_i, f(x_i | w, b)) \right)$$

user-defined
cost matrix

Loss Function	Has Cancer	Doesn't Have Cancer
Predicts Cancer	0	1
Predicts No Cancer	1000	0

[table from Yisong Yue]